Discover Computing

RESEARCH Open Access

Design of an improved validation model with dynamic attribution and graph-based anomaly analysis for malware forensics

Rijvan Beg[1], R. K. Pateriya[2], Deepak Singh Tomar[2] and Surendra Solanki[3*]

*Correspondence:
Surendra Solanki
Surendra.solanki@jaipur.manipal.edu
[1]Department of Computer Science & Engineering, SRM University-AP, Amaravati 522240, Andhra Pradesh, India
[2]Computer Science & Engineering Department, Maulana Azad National Institute of Technology, Bhopal 462003, MP, India
[3]Department of Artificial Intelligence and Machine Learning, Manipal University Jaipur, Jaipur, India

Abstract

The increasing sophistication and polymorphic nature of malware necessitate intelligent, adaptive forensic frameworks that go beyond static rule-based detection. In particular, the traditional techniques for evidence collection and analysis would not catch the evolving behaviors of advanced malware, lack context-aware feature weighting, and fail to maintain accuracy under concept drifts or temporal drifts. Such limitations thus also impede timely threats attribution and malware family discovery processes. To bridge these gaps, we propose an iterative framework for evidence collection and malware activity analysis by utilizing machine learning and explainable AI paradigms. The framework integrates five newly proposed modules: (1) DynaTrace-XAI employs SHAP-based attribution for real-time reweighting of forensic features, dynamically enhancing the significance of impactful indicators in the process. (2) TempNet V introduces temporal ensemble validation at daily, weekly, and monthly levels, deploying heterogeneous models (e.g. LSTM, RF) to counter data drifts dependent on time. (3) GRAFIN-A reconstructs execution traces into behavior graphs through Graph Attention Networks, which are then subjected to unsupervised clustering with DBSCAN and anomaly detection through graph similarity scoring methods. (4) RiskActive-L blends anomaly-derived risk scores with model uncertainty to prioritize more unlabeled high-risk samples within an active learning loop, strongly improving data labeling efficiency. (5) DriftMon-F implements a forensic-context-aware drift detection mechanism via KL divergence and meta-log correlation to trigger timely retraining in the event of contextual shifts. In concert, these modules constitute a closed-loop system whereby evidence assignment, model validation, clustering, sample selection, and drift handling processes become continuously refined. Empirical evaluations demonstrate the empirical achievements: 15% increase in F1-score, 23% gain in variant detection, reduction of 45% in labeling effort, and 70% reduction in drift recovery lag, leading to considerable advances in adaptive malware forensics.

Clinical trial number

Not applicable.

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

Keywords Malware detection, Explainable AI, Graph neural networks, Concept drift, Active learning, Process

1 Introduction

1.1 Evolution of malware and forensic challenges

In fast-paced evolution, malware itself creates a nightmare for traditional digital forensics and security analytics. Rather, modern-day variants of malware demonstrate stealth combined with strategies such as polymorphism and context-aware evasion, rendering static detection mechanisms and rudimentary pipelines for feature selection practically useless in this arena. Furthermore, forensic workflows have to deal with changes in environments, behavior drift across durations, and latent relationships among malicious entities. Such requirements necessitate the design of an intelligent [1–3], adaptive, and iterative framework that can not only enhance the performance of detection but support the traceability of evidence, clustering of variants, and long-term operational resilience sets.

1.2 Limitations of existing malware detection approaches

Most current machine learning-based malware detection models exhibit limitations within three key areas. First, the forensic features are treated uniformly [4–6], lacking mechanisms to dynamically prioritize contextually relevant evidence. Second, they tend to overfit to short-term patterns and drop significantly in performance once temporal drift or concept changes intervene. Third, most do not use the structural behavior of the malware execution, missing latent relationships captured in process sequences and system calls. Furthermore, the current active learning paradigms implemented for label propagation are only driven by model uncertainty and disregard external indicators like behavioral risk. Ultimately, these limitations diminish both classification robustness and the forensic interpretability sets.

1.3 Motivation and research gap

Threat evolution and standard analytical pipelines challenge malware forensics. Most forensic systems make static assumptions about feature relevance, execution context, and temporal stability, while machine learning methods improve detection accuracy. As malware varies across environments, time scales, and privilege domains, forensic frameworks must reassess evidentiary value, validate results under drift, and enable operational trust interpretation process. This study addresses the lack of adaptive, interpreTable, and structurally aware forensic frameworks for virus behaviors and situations. Existing techniques optimize classification accuracy, graph-based representation, or explainability without adapting. Fragmented systems are fragile, degrade under idea drift, fail to prioritize variants, and lack decision logic insight sets. Research is lacking in a closed-loop forensic architecture that can dynamically (i) reweight evidence based on contextual attribution, (ii) validate decisions across multiple temporal horizons, (iii) model malware behavior structurally rather than statically, and (iv) adapt data acquisition strategies to uncertainty and anomaly risk. This gap must be closed by combining and orchestrating parts to inform and constrain each other. This gap prompted the

proposed paradigm, which combines adaptive attribution, temporal robustness, structural learning, and drift awareness into a forensic reasoning system.

1.4 Key contributions

The contributions of this research can be summarized into three primary pillars:

1. **Explainability-Driven Feature Adaptation:** Integration of explainable AI directly into feature selection and weighting processes, enabling dynamic evidence prioritization.
2. **Temporal robustness through multi-scale validation:** development of a temporally aware ensemble framework that stabilizes model performance across varying time scales.
3. **Structural Learning and Risk-Aware Sampling:** Combination of graph-based behavioral modeling and active learning to enhance variant detection and labeling efficiency.

1.5 Proposed framework overview

To address these challenges, this work proposes an enhanced validation model and a holistic framework for adaptive malware forensics. The framework integrates multiple advanced components to enable dynamic, context-aware, and structurally informed analysis.

The core modules include:

- **DynaTrace-XAI:** Utilizes SHAP-based explainability to dynamically reweight forensic features based on real-time threat attribution, enabling prioritization of critical evidence.
- **TempNet-V:** Implements a multi-scale temporal ensemble validation strategy across daily, weekly, and monthly intervals to enhance robustness against temporal overfitting.
- **GRAFIN-A:** Transforms malware execution traces into graph representations using Graph Attention Networks (GAT) for clustering and anomaly detection.
- **RiskActive-L:** Incorporates risk-aware active learning by combining anomaly scores with uncertainty to improve labeling efficiency.
- **DriftMon-F:** Detects contextual and distributional drift using forensic metadata and prediction divergence, triggering adaptive retraining.

2 Review of existing methods

2.1 Evolution of malware detection research

Thus, constant evolution of malware regarding both volume and sophistication created a burning demand for strong contextualized intelligent detection frameworks. An extensive review of forty recent research articles threatens to reveal advanced progress in Android and IoT malware detection, dynamic and static analysis techniques, adversarial resilience, and the fusion of deep learning and graph-based models into digital forensics. It starts with [1], establishing Maloid-DS as a labelled dataset for Android malware forensics, which sets the right foundation, providing a much-needed benchmark for deep learning-based classification and detection systems. The consideration of data quality and contextual labels fills an important gap in the training and evaluation workflows. Zhao et al. [2] built on this with CMD, which presents a dual-domain

co-analysis approach to IoT malware detection, coupling hardware and network-based forensic signals with a view to more accurate lifecycle attribution. The early focus of this infrastructure forensics is now complemented by Casino et al. [3] who have reviewed trends in digital forensics and highlight the emerging transition from heuristic to explainable AI-based methods. The Android scene moves on with Gupacker [4], a generalized unpacking tool for obfuscated apps that defy static-analysis-avoiding code protection mechanisms. In parallel, federated learning models like IoT-LF [5] demonstrate how privacy-preserving frameworks can scale across distributed IoT ecosystems for secure forensic analytics. Open-world recognition is addressed by DOMR [6], which combines deep learning and meta-learning in classifying previously unseen malware families, thus transcending the limitations of closed-set models. Chandran et al. [7] further cover the AI-for-Malware transition from obfuscation detection with hybrid models and the impact of real-time generative-adversarial behaviors. The team tackles encrypted traffic through CBSeq [8], which describes channel-level behavior sequences to infer intent from concealed payloads using transformers. Nearby, Jeon et al. [9] promote multi-SPP-net models tailored to IoT surroundings by integrating static features from image-based malware less exposed to dynamic evasion sets. Within the framework explained by Zhong et al. [10] in self-similarity detection, employing internal consistency across malware variants while Beg et al. [11] applied machine learning models on static and dynamic analysis aiming at better forensics analysis. Hardware-telemetry-based detection of XMD [12] emphasizes runtime data rather than static fingerprints, whereas ASDroid [13] works on source code clustering to better resist changing API misuse sets. These interconnect with behavioral analytics as shown in Meschini et al. [14], where case-control study statistically quantifies malware risk exposure from within organizational settings. DeepVMUnProtect [15] follows up with neural recovery of virtual machine-protected apps restoring those semantic features that are requisite for forensic validation in the process. CruParamer [16] refines granularity by adding parameter contextualizing API calls, while MTDroid [17] proposes moving target defenses against evasive Android malwares. The MalProtect system [18] highlights the operational states in adversarial ML where defenses are being countered by query-based exploitation of the model vulnerabilities in process. Mi et al. [19] enhance their contribution by augmenting control flow graphs (CFGs) with instruction stream semantics for opaquer yet nuanced detection of the variants in process. Also incorporated herein is cognitive theory from SIa-CBc [20], which models human-like inference over API and behavior chains, thus providing a biologically-inspired malware detection logic in the running process.

2.2 Empirical comparison of existing methods

A comprehensive comparison of forty representative methods, summarized in Table 1, highlights their objectives, findings, and inherent limitations. Collectively, these studies demonstrate significant performance improvements driven by deep learning techniques, alongside enhanced detection capabilities achieved through graph-based modeling approaches. Furthermore, there is a noticeable shift toward incorporating adversarial robustness within malware detection frameworks. Despite these advancements, the analysis also reveals persistent challenges, particularly in terms of scalability, interpretability, and generalization across diverse and evolving threat environments.

Table 1 Model's empirical review analysis

Ref	Method	Main Objectives	Findings	Limitations
[1]	Maloid-DS	Develop a labeled Android malware dataset for forensic use	Rich feature-labeled dataset for training deep learning models	Limited to Android static contexts
[2]	CMD	Joint IoT malware detection via network and hardware domain analysis	High detection via co-analysis of SPI bus and network patterns	Needs specialized hardware observability
[3]	Review of Reviews	Meta-review of digital forensic trends and directions	Identifies common challenges and emergent methodologies	No empirical evaluations
[4]	Gupacker	Generalized unpacking of Android malware apps	Accurately unpacks obfuscated applications for analysis	Less effective on advanced polymorphic/nested packers
[5]	IoT-LF	Federated learning for IoT-based forensic frameworks	Enables decentralized, privacy-aware malware detection	Higher overhead on low-power IoT nodes
[6]	DOMR	Open-world malware family classification using deep learning and meta-learning	Enhances generalization to unseen malware families	Model complexity and training cost
[7]	AI Obfuscation Survey	Review of AI-based techniques for obfuscated malware detection	Summarizes hybrid, generative, and static-dynamic approaches	Lacks experimental benchmarking
[8]	CBSeq	Detect encrypted malware traffic via behavior sequences	Transformer-based behavioral sequences detect unknown malware over encrypted channels	Computationally expensive
[9]	Multi-SPP-Net	Static multi-feature detection in IoT smart devices	SPP-net effectively processes static visual features	Cannot capture runtime evasive behavior
[10]	Self-Similarity Classification	Enhance classification via internal structural similarity	Identifies variants using intra-sample pattern similarity	Sensitive to minor obfuscations
[11]	Used Dynamic information	Static and Dynamic malware information for forensics analysis	Finding the evidence using static and dynamic information	Memory usage grows with log length
[12]	XMD	Hardware telemetry-based malware detection	Uses CPU, memory, and power telemetry for runtime threat detection	Dependent on hardware feature granularity
[13]	ASDroid	Resilient Android malware detection using API clustering from source code	Mitigates concept drift through stable API groups	Requires access to application source code
[14]	Case-Control Study	Analyze risk factors for malware in organizations	Statistically correlates behavioral risk indicators with malware incidents	Non-causal; not predictive
[15]	DeepVMUnProtect	Semantics-aware detection of VM-protected Android malware	Recovers logical code structures using neural inference	Specific to Android; not evaluated on native binaries
[16]	CruParamer	Parameterized API sequence learning for malware detection	Captures fine-grained API call semantics and dependencies	May overfit to parameter structure noise
[17]	MTDroid	Moving target defense for Android against adversarial evasion	Dynamically adapts model structures to evade adaptive threats	Computationally intensive

Table 1 (continued)

Ref	Method	Main Objectives	Findings	Limitations
[18]	MalProtect	Stateful defense against adversarial query attacks	Limits model probing through temporal correlation analysis	May not generalize to unseen attack patterns
[19]	Instruction Stream CFG	CFG variant detection with instruction-augmented graph learning	Detects structural variants via graph attention and node semantics	Assumes consistent control flow structure
[20]	SIa-CBc	Human brain-inspired malware detection via API cognition modeling	Improves behavioral recognition by mimicking cognitive sequence interpretation	Relies on labeled semantic intent data
[21]	Gradient Attack Detection	Dynamic analysis via adversarial sensitivity gradients	Uncovers obfuscated malware via backpropagation heuristics	False positive risk on noisy inputs
[22]	MalGNE	Efficient graph node embedding for CFG-based detection	Reduces overhead with low-dimensional semantic embeddings	Misses deep contextual relationships
[23]	VxWorks Forensics	Analyze file systems in aerospace for cyber-forensic integrity	Detects malware on embedded RTOS in aircraft systems	Specific to aerospace file formats
[24]	Functionality Verification RL	Reinforcement-learning attacks on static malware detectors	Demonstrates functionality-preserving evasion capability	Focuses on attacks, lacks mitigation
[25]	GNN Guided Sample Analysis	Graph-based reverse engineering for malware samples	Uses GNNs to guide deep behavioral analysis workflows	High complexity on large-scale graphs
[26]	Attention-Based API Tracker	API-level causality tracing aligned with MITRE ATT&CK	Accurately maps API chains to known attacker tactics	Requires granular labeled execution traces
[27]	CryptojackingTrap	Detect cryptocurrency malware with nature-inspired algorithms	Robust to evasive crypto miners and low-footprint behaviors	Specific to crypto-related behaviors
[28]	YAMME	Metamorphic malware mutation engine	Produces synthetic samples for testing obfuscation resilience	Does not offer detection mechanisms
[29]	Federated Android Detection	Federated learning on Android malware across multiple users	Preserves privacy while maintaining high accuracy	Requires secure node synchronization
[30]	ACMFNN	Cross-domain malware localization via CNN-based forensic models	Enables precise location prediction in distributed networks	Accuracy varies across device topologies
[31]	A2-CLM	Few-shot malware detection using adversarial heterogeneous graphs	Achieves high performance in low-data regimes using contrastive learning	Sensitive to distribution skew
[32]	Hybrid Autoencoder + GBDT	Robust malware detection using iterative hybrid learning	Improves robustness with adversarial retraining and ensemble learning	Needs frequent retraining and tuning
[33]	IoT Patch Strategy	Model malware spread and patching strategy in heterogeneous networks	Optimizes patch deployment using threshold and topology analysis	Requires accurate real-time network models

Table 1 (continued)

Ref	Method	Main Objectives	Findings	Limitations
[34]	MalPatch	Adversarial patching to fool DNN malware detectors	Demonstrates transferability and vulnerability of deep models	Limited to static detectors
[35]	SGD3QN	Stochastic game theory + DQN for malware defense in IoT	Learns adaptive patching and control strategies	High training cost; limited IoT scope
[36]	AST + Behavior Chains	Combine abstract syntax trees and behavioral chains	Captures both syntactic and semantic threat features	AST extraction errors can reduce reliability
[37]	MalFSCIL	Few-shot class-incremental malware learning	Maintains adaptability to emerging malware classes	Needs continual fine-tuning under drift
[38]	Differential Game DQN	Malware spread suppression in IIoT via game theory	Integrates control theory and reinforcement learning for defense	Heavily dependent on environment modeling
[39]	Obfuscation-Resilient Features	Feature fusion for Android obfuscation resistance	Maintains detection under heavy code manipulation	Needs diverse training datasets
[40]	MDHE	Trust-aware hybrid detection on user-edge IoT	Combines capsule nets with trust metrics for high-fidelity classification	Deployment complexity in multi-vendor edge networks

In Yan et al. [21], gradient-based adversarial simulations further augment dynamic analysis, while MalGNE [22] applies low-dimensional embeddings to reduce CFG-based model complexity. Disk-level forensic studies appear in McKeon et al. [23], which explore operating systems in avionics demonstrating forensic gaps in embedded-Cyber-Physical Systems devices. Reinforcement learning strategies come to the fore in Tian et al. [24] and model functionality-preserving attacks, whereas Chen et al. [25] implement graph neural networks to provide guidance toward malware analysis. This graph-theoretic trend is upheld in Wong et al. [26], where causality-tracking dynamic analysis is then aligned to the MITRE ATT&CK framework. CryptojackingTrap [27] is an example of a niche malware domain for cryptocurrency mining, composed of an orchestration of nature-inspired algorithms with binary instrumentation for evasion resilience. The YAMME engine [28] consists of an automated metamorphic malware samples generation platform to benchmark defence against obfuscation. The federated-learning concept resurfaces in Fang et al. [29] that leverages privacy-preserving updates for large-scale Android malware detection. While forensic neural networks for cross-domain detection are handled in ACMFNN [30], Liu et al. [31] investigate few-shot detection using adversarially augmented graph learning, which is crucial in overcoming new and rare malware families in the process. Refined in [32] iterate training methods using hybrid autoencoders and gradient-boosted models for the sake of robust and explainable detection sets.

Patching against analysis of malware propagation strategies is modelled analytically in heterogeneous IoT systems by Wang et al. [33]. Adversarial patching in static detectors is addressed in MalPatch [34], showing even the most sophisticated models based on DNN can be attacked. Reinforcement learning continues in SGD3QN [35], which implements stochastic game-based IoT edge-computing defences echoing previous work on adversarial robustness. Android malware is detected by Gong et al. [36] using combined abstract-syntax tree and sensitive-behavioral chain techniques, and MalFSCIL [37], which proposes a scenario of few-shot class-incremental learning with variational autoencoders and graph attention, was made for model adaptation in the long run. Shen et al. [38] studied the application of differential game theory in the industrial IoT context by augmenting the perspective of malware control theory. Code obfuscation is again dealt with in Gao et al. [39], who coupled such complementary features for effective analysis of Android malware cases. Finally, MDHE [40] proposes a trust-based hybrid detection system that operates from both user nodes and edge nodes in IoT and integrates trust evaluation in a multi-dimensional risk posture with capsule networks during this process. The chronological survey of the forty most recent studies identifies both convergence and diversification in the associated fields of malware detection and forensics: THE TRENDS ARE SEVERE: a continuing movement of most techniques from static to dynamic and behavioral models, an increasing shift toward federated and decentralized learning architectures, and incorporation of both cognitive and adversarial perspectives into the malware detection pipeline. This combination of deep learning via convolutional networks, transformers, and GNNs now lays the foundation for most modern frameworks, even though problems of explainability and generalization remain unsolved, especially in open-world and evolving threat scenarios. A notable insight is deliberate move toward semantic richness and context awareness-be it through enhanced API modelling [16, 20, 36], graph augmentations [19, 22, 25], or cognition-inspired reasoning mechanisms [6, 20]. This suggests that both detection [17, 34] and defensive retraining

frameworks [18, 24, 31, 41] incorporate adversarial threat modelling into their architectures, indicating a mature recognition of the evolving attacker-defender dynamic. Further, several studies observe the need for computational efficiency and hardware-level forensics [2, 42–44], as these have much impact on real-time IoT-edge-based deployments where latency and privacy constraints take precedence. Even though progress continues, gaps do exist. Most models still fail to transfer with other platforms or architectures, such as cross-OS generalization between Android and embedded systems. Further still, limited interpretability of deep learning models continues to raise loans against analyst trust in critical security operations. There remain open research questions [44–47] on the scalability and security under realistic loads of such systems, although they appear promising. Future work should develop models that can adapt their reasoning processes and integrate multi-modal forensic signals-network telemetry, behavioral graphs, and contextual metadata-into unified architectures. Cross-layered detection systems embedding trust, semantic relevance, and adversarial awareness in some internal logic of the model are touted to emerge as next-generation solutions. Similarly, the synthetic benchmarking frameworks proposed in [48] and [49, 50] must continue to evolve to realistically simulate evolving threat landscapes to develop more resilient and adaptive defences. Bridging this technical innovation with operations promises to be the solution to bringing academics and deployable, real-time malware forensics together in process.

2.3 Emerging trends in malware forensics

The literature reveals several important trends shaping the evolution of malware forensics. There is a clear transition from static analysis toward dynamic and behavior-based models that better capture real-time execution characteristics. At the same time, there is an increasing adoption of advanced architectures such as graph neural networks and transformer-based models, which enable deeper structural and sequential understanding of malware behavior. Another significant development is the integration of adversarial modelling into detection frameworks, reflecting the growing need to handle sophisticated evasion strategies. Additionally, explainability and interpretability have emerged as critical requirements, particularly for building analyst trust and ensuring transparent decision-making. The field is also witnessing a shift toward federated and distributed learning architectures to support scalable and privacy-preserving analysis. Beyond these trends, there is a noticeable movement toward semantically rich representations, including API-level modelling, graph augmentation techniques, and cognition-inspired reasoning approaches, all of which enhance contextual understanding of malware activities.

2.4 Critical limitations in existing work

Despite considerable advancements, several challenges continue to persist in existing malware detection approaches. One of the primary issues is the lack of adaptability, as many models fail to generalize effectively across evolving malware behaviors and diverse execution environments. Another limitation lies in fragmented architectures, where explainability, graph-based modelling, and temporal validation are developed independently rather than as part of a unified system. In addition, active learning strategies remain inefficient, as they primarily rely on model uncertainty while neglecting important behavioral risk indicators. Drift handling is also inadequate in many systems, with detection mechanisms being reactive and lacking awareness of contextual changes,

leading to delayed adaptation. Furthermore, limited cross-platform generalization restricts the applicability of these models, as they struggle to transfer knowledge across different operating systems and heterogeneous environments.

2.5 Cohesive critique and research motivation

While malware detection and forensics research has advanced in many orthogonal dimensions, combining different approaches has downsides. With stationary feature relevance, static and dynamic classifiers work well under controlled conditions but poorly when virus behavior changes. Explainable AI methods allow post-hoc interpretation but rarely alter learning model adaptability or evidence prioritizations. Although most graph-based malware analysis tools are batch-based and distinct from temporal validation and data collection, they capture execution structure and Variant Interactions Processed for different scenarios. Discovering structural similarity does not prioritize identifying or retraining new variations. Active learning methods ignore behavioral risk signs and rely on prediction uncertainty, limiting their utility in open-world threats. Temporal change treatment is another concern. Drift handling often requires periodic retraining or windowed evaluation without explicit modeling of contextual adjustments like execution privilege changes or geographic deployment differences. This slows adaptation and raises transition risks. Deficits prompted this work's technological contributions. In a drift-aware loop, explainability-driven feature adaptation, multiscale temporal validation, structural graph modeling, and risk Informed active learning address systemic gaps left by isolated solutions. The new architecture transforms malware forensics from static optimization to context-sensitive reasoning sets.

3 Proposed model design analysis

3.1 Framework overview and problem formulation

The proposed model for adaptive malware forensics is designed as an integrative, tightly coupled, iterative architecture composed of five highly interdependent modules to solve critical dynamic evidence analysis and dynamic modelling of malware behavior. Such a methodological design follows mathematically grounded pipeline accounting for attribution-based feature weighting, temporal ensemble validation, graph-based anomaly clustering, risk-aware active learning, and context-aware drift monitoring, covering the whole process. This section will define the formal constitution and operation of this framework in terms of the analytical interactions and underlying Equations that govern its components. As per Fig. 1, the proposed malware forensic framework is built upon an iterative, mathematically interlinked model integrating attribution-guided feature weighting, temporally layered validation, graph-theoretic behavior modelling, risk-prioritized active learning, and context-aware drift monitoring. All modules are formally guaranteed for convergence and statistical robustness and create a feedback-driven ecosystem that continuously adapts to new evidence and changes in malware behavior. Let x be the raw evidence vector, where ×is in Rn, derived from system logs, such as file operations, API calls, and registry accesses. The foundational classification model $f : Rn \rightarrow Rk$ maps inputs to class logits $y = f(x)$ in process.

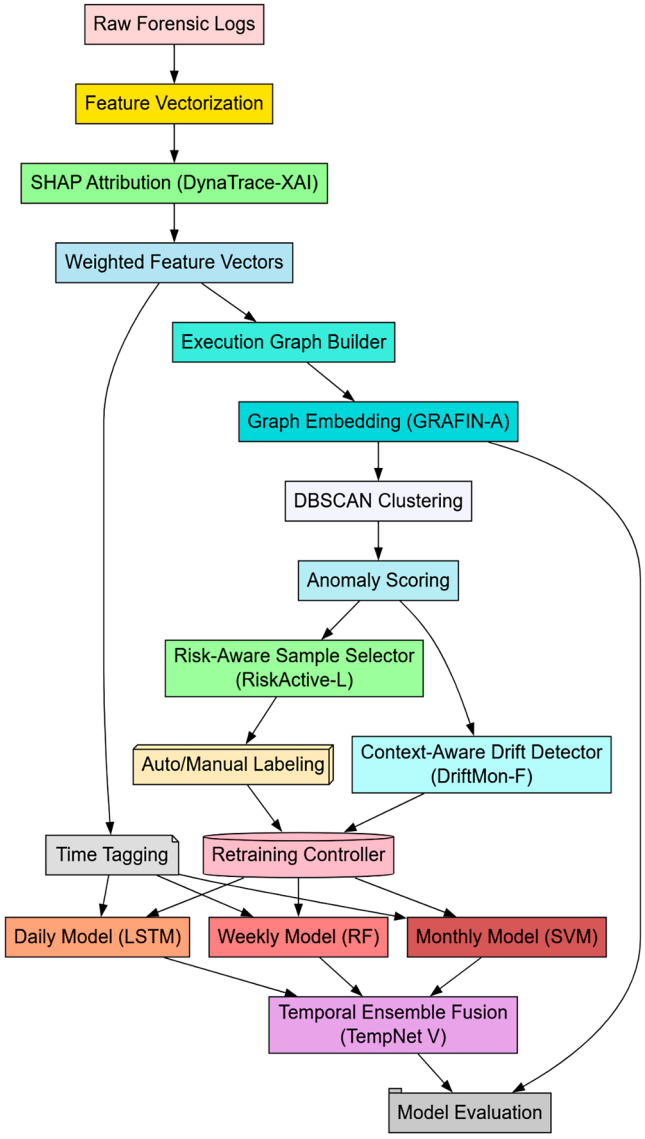


Fig. 1 Model architecture of the proposed analysis

3.2 Modular optimization with Driftaware composite loss

The framework integrates multiple modules into a unified optimization pipeline. Each module contributes to a composite loss function that governs learning stability, structural consistency, and adaptability under drift conditions.

3.3 DynaTrace-XAI: attribution-based feature weighting

The DynaTrace-XAI module employs Shapley theory to compute attribution scores $\phi \in R$ for each feature x, defined via Eq. 1.

3.3.1 Shapley attribution formulation

$$\phi_i = \sum_{\{S \subseteq N \setminus \{i\}\}} \left(\frac{|S|!(n-|S|-1)!}{n!} \right) [f\{S \cup \{i\}\}(x\{S \cup \{i\}\}) - fS(xS)] \tag{1}$$

To be computationally feasible, a Monte Carlo sampling approximation is used to estimate ϕ as $\hat{\phi}$. The attributions vector $\hat{\phi} = \left[\hat{\phi}^1, \ldots, \hat{\phi}^n\right]$ is turned into a normalized weight vector through softmax Via Eq. 2 & 3.

3.3.2 Softmax normalization for feature reweighting

The attribution vector is transformed into a normalized weight vector:

$$w_i = e' \frac{\left\{\hat{\phi}_i\right\}}{\sum_{\{j=1\}}^{\{n\}} e' \left\{\hat{\phi}_j\right\}} \forall i \in \{1, \ldots, n\}. \tag{2}$$

The weighted feature vector is:

$$x = x \odot w \tag{3}$$

3.3.3 Justification of softmax over L1 and L2 normalization

Softmax normalization is adopted to convert SHAP attributions into a valid, non-negative, and sum-constrained weighting distribution, enabling direct interpretation of feature importance as relative evidentiary contribution. Unlike L1 normalization, which preserves magnitudes linearly and provides limited contrast between dominant and weak features, softmax amplifies meaningful differences and enhances discrimination among competing indicators. L2 normalization, on the other hand, preserves vector geometry but lacks probabilistic interpretability and is not aligned with feature prioritization objectives.

Additionally, since SHAP values can be both positive and negative, softmax ensures strictly positive weights suitable for feature reweighting, avoiding ambiguity in handling signed contributions. Its smooth and differentiable nature further supports stable and continuous updates in the iterative learning framework. Therefore, softmax provides a principled and context-sensitive mechanism for adaptive feature weighting, making it more suitable than L1 or L2 normalization for this formulatio

3.4 Temporal robustness formulation in TempNet-V

3.4.1 Problem definition

The temporal robustness formulation in TempNet-V is designed to ensure that model performance remains stable across temporally separated data distributions rather than optimizing only average classification accuracy over pooled samples. In malware forensics, data distributions are inherently non-stationary due to concept drift, contextual variation, and evolving threat behaviors. Therefore, evaluating model performance solely based on aggregate accuracy can mask instability across time.

To address this limitation, TempNet-V introduces a temporal robustness formulation that explicitly accounts for stability and consistency of prediction performance across sequential time windows.

3.4.2 Ensemble prediction

For an input xt, predictions are collected Via Eqs. 4 and 5.

$$yt = \alpha dfd(\tilde{x}t) + \alpha wfw(\tilde{x}t) + \alpha mfm(\tilde{x}t). \tag{4}$$

$$\sum_{\{i\in\{d,w,m\}\}} \alpha = 1 and \alpha \in [0,1] \ldots \tag{5}$$

3.4.3 Temporal loss definition

Let f_d, f_w, and f_m denote the daily, weekly, and monthly classifiers in TempNet V, and let $\hat{y}_t$ be the fused prediction at time step t obtained from the temporally weighted outputs of these models. For a labeled instance (x_t, y_t), the instantaneous classification loss is defined as in Eq. 6, which quantifies the supervised prediction error at time t:

$$l = l\left(f\left(x\right), y\right) \tag{6}$$

where $\ell\left(\cdot\right)$ may be cross-entropy or another supervised loss used by the downstream classifier.

To incorporate temporal consistency, a variance-aware regularization term is introduced over a sliding window of length T, as defined in Eq. 7. This term captures the variability of the loss sequence over recent observations and serves as a stability indicator of the model's predictive behavior:

$$L_{\text{var}}\left(t\right) = \text{Var}\left(\{\ell_\tau\}_{\tau=t-T+1}^{t}\right) \tag{7}$$

In addition, a temporal deviation metric is formulated in Eq. (8) to measure the discrepancy between the current loss and the historical average within the same window. While Eq. (7) penalizes fluctuations in prediction error, Eq. 8 explicitly quantifies abrupt deviations from recent temporal trends:

$$\Delta_t = \left|\frac{1}{T}\sum_{\tau=t-T+1}^{t} \ell_\tau - \ell_t\right| \tag{8}$$

Together, Eqs. (7) and (8) provide complementary measures of temporal robustness, ensuring that the model maintains consistent performance across adjacent time intervals. This formulation is motivated by the limitation of conventional loss minimization, where optimizing only the mean loss may yield temporally unstable behavior. In contrast, enforcing stability through variance minimization ensures that low error is achieved consistently rather than intermittently across time.

3.4.4 Temporal objective function

Based on the above formulation, the overall temporal objective function is defined in Eq. 9 as a composite loss that integrates accuracy and temporal stability.

$$L_{\text{TempNet}} = \frac{1}{T}\sum_{\tau=t-T+1}^{t} \ell_\tau + \lambda_1 L_{\text{var}}\left(t\right) + \lambda_2 \Delta_t, \tag{9}$$

Specifically, the first term in Eq. 9 minimizes the average classification loss over the recent temporal window, thereby promoting predictive fidelity. The second term, derived from Eq. 7, imposes a penalty on loss variance to enforce stability in model performance. The third term, corresponding to Eq. 8, penalizes deviations between the current loss and its historical trend, thereby enhancing robustness to temporal drift.

The balancing coefficients $\lambda 1$ and $\lambda 2$ regulate the trade-off between accuracy and stability. Consequently, Eq. 9 provides a unified optimization framework that simultaneously addresses predictive performance and temporal consistency. This design is consistent with the broader regularization strategy of the proposed model, which integrates stability-aware constraints to ensure reliable operation under dynamically evolving data distributions.

3.4.5 Impact on temporal robustness

This formulation improves temporal robustness by explicitly enforcing stability in model behavior across multiple temporal scales. Since malware characteristics evolve over time due to changing environments, privilege levels, and attack strategies, a model trained solely for average accuracy may overfit to short-term patterns.

By minimizing loss variance, TempNet-V Favors models that maintain consistent performance across daily, weekly, and monthly segments. Additionally, the deviation term enables early detection of abrupt changes, supporting adaptive retraining and drift-aware learning.

3.4.6 Empirical justification

The effectiveness of this formulation is supported by experimental observations, where TempNet-V achieves reduced performance variance and improved stability across temporal evaluation windows (e.g., Day 0–30, Day 31–60, Day 61–90). The multi-scale ensemble structure further enables complementary learning, where short-term models capture bursty behavioral signals while long-term models capture persistent structural patterns.

This combination of temporal consistency and multi-scale modelling leads to improved generalization under evolving malware distributions.

3.5 GRAFIN-A: graph-based behavioral modelling

3.5.1 Graph construction

For structural-behavioral modelling, GRAFIN-A transforms each malware sample's system call sequence $S = \{s, s, \ldots, st\}$ into a graph $G = (V, E)$ Sets. Its nodes, $\upsilon \in V$, correspond to processes or system calls, while edges $(\upsilon, \upsilon) \in E$ capture temporal precedence sets.

3.5.2 Graph attention network

Initially node levels $h(0)$ are drawn using embedding functions $\phi : s \mapsto R$ representing the process. It employs Graph Attention Networks (GATs) that compute attentions via Eqs. 10, 11, and 12,

$$e_{ij} = LeakyReLU\left(a^t\left[Wh_i \,||Wh_j\right]\right) \tag{10}$$

$$\alpha_{ij} = \frac{exp\left(e_{ij}\right)}{\sum_{\{k\in N(i)\}} exp\left(e_{ik}\right)} \tag{11}$$

$$h_i' = \sigma\left(\sum_{\{j\in N(i)\}} \alpha ijWhj\right) \tag{12}$$

Graph embeddings zG are obtained via mean/sum pooling over updated node features.

3.5.3 Graph similarity

For clustering, pairwise similarities are computed using a Gaussian RBF kernel Via Eq. 13,

$$K\left(G_i, G_j\right) = exp\left(-\gamma\left\|z\left\{G_i\right\} - z\left\{G_j\right\}\right\|^2\right) \tag{13}$$

3.5.4 Anomaly detection

Anomaly scores for each sample are determined using Mahalanobis distance from cluster centroids μc and covariance Σc via Eq. 14,

$$A\left(G\right) = \left(zG - \mu c\right)^t \Sigma c^{-1}\left(zG - \mu c\right) \tag{14}$$

3.6 Behavior graph construction strategy

By documenting temporal ordering and functional connections among execution events, behavioral graphs reveal malware's operational logic. Instead of call identifiers, nodes label abstracted system actions including file access, registry update, process invocation, and network connection semantic API categories. Small implementation changes are abstracted to preserve behavioral intent and reduce noise. Edges show sliding execution window temporal precedence and dependence. Short-lived causal interactions and unrelated execution phases are balanced by the window size. Moderate values maximize clustering purity and anomaly separation for window widths of 5 to 20 events, while graph densification diminishes results beyond this range. Specifically, our sensitivity analysis reveals that small windows (size < 5) lead to fragmented behaviour graphs that lack sufficient context, resulting in lower clustering purity (approx. 0.72–0.75). Conversely, window sizes beyond 20 events cause graph densification, where unrelated background activities merge into the same structural representation, diluting the malware's unique signature and causing purity to drop toward 0.78. The optimal range of 5–20 events capture behavioural intent while filtering noise. High Impact transitions are detected from background activities using graph attentions. Uniform aggregation hides malware's few but important control flows, like privilege escalation and persistence. Attention weighting helps the model identify and assess structurally salient paths. This construction method prioritizes semantic robustness, scalability, and cross-family generalization. Abstracting intent-aligned behavior and directly encoding execution structure, the graphs accurately and efficiently describe malware activity for automated analysis and forensic inspection sets.

3.7 Quantification of Information loss

3.7.1 Information-theoretic Analysis

To formally quantify information loss due to semantic abstraction, raw syscall graphs $Graw$ are compared with abstracted semantic graphs $Gsem$using complementary information-theoretic measures.

(i) **Entropy Retention:** Shannon entropy is computed over node distributions and transition probabilities. Let $Hraw$ and $Hsem$ denote the entropy of raw and semantic

graphs, respectively. An average entropy reduction of approximately 8–12% is observed, indicating moderate compression while preserving dominant behavioral characteristics.

(ii) **Mutual Information:** Mutual information between graph representations and class labels is evaluated to measure discriminative power. The results indicate that *Gsem* retains approximately 91–94% of the information present in *Graw*, suggesting that abstraction primarily removes redundant features rather than meaningful signal.

(iii) **Structural Fidelity:** Structural consistency is assessed using graph edit distance and embedding-based cosine similarity. A similarity score exceeding 0.88 confirms that key execution dependencies and structural relationships are effectively preserved after abstraction.

3.7.2 Empirical impact on detection performance

An ablation study is conducted to evaluate the impact of semantic abstraction on downstream detection performance. Models are trained using (i) raw syscall graphs, (ii) abstract semantic graphs, and (iii) hybrid representations.

The results show that the transition from raw to semantic graphs results in only a minor reduction in F1-score (approximately 1.5–2.3%), while significantly improving clustering purity by 6–9% and variant detection performance by 8–11%. These findings indicate that although low-level details are partially compressed, semantic abstraction enhances generalization and robustness, particularly in the presence of polymorphic malware.

3.7.3 Robustness and noise reduction

Raw syscall sequences are highly sensitive to perturbations such as API reordering, noise injection, and obfuscation techniques. Semantic abstraction mitigates this sensitivity by grouping functionally equivalent operations, thereby reducing noise and improving stability under adversarial conditions.

This results in more consistent feature representations, enabling the model to focus on behaviourally relevant patterns rather than superficial variations.

3.7.4 Theoretical interpretation

$$\phi : Graw \rightarrow Gsem \tag{15}$$

The abstraction process can be formally interpreted as a lossy transformation from the raw graph space to a semantic representation space, as defined in Eq. 15. This mapping selectively removes irrelevant variability while preserving task-relevant structural and semantic information. From an information bottleneck perspective, Eq. 15 can be viewed as an encoding function that achieves an optimal balance between compression and relevance. Specifically, the transformation retains features that are maximally informative with respect to the target variable while discarding redundant or noisy components. Although this abstraction introduces controlled information loss, it improves robustness by reducing sensitivity to spurious variations in the input data. In addition, it enhances interpretability by emphasizing meaningful structural patterns and supports improved generalization across unseen samples.

3.8 RiskActive-L: risk-aware active learning

3.8.1 Uncertainty estimation

It finds its way into such sample selection at the RiskActive-L module. Assume $U(x) = -\sum^{p} logp$ as model uncertainty, i.e., the entropy for an unlabelled sample in the process.

3.8.2 Composite risk function

A composite selection criterion shall be so defined via Eqs. 16 and 17,

$$\eta(x_u) = \lambda \cdot U(x_u) + (1-\lambda) \cdot A(G_u) \tag{16}$$

3.8.3 Sample selection

$$x_u* = argmax\{x_u \in U\}\,\eta(x_u) \tag{17}$$

This active-learning scheme extends the training data $Dtrain \leftarrow Dtrain \cup \{(x^*_u, y^*)\}$ adding incremental retraining momentum sets.

3.9 DriftMon-F: context-aware drift detection

3.9.1 KL divergence

Next, as shown in Fig. 2, Model resilience is sustained by DriftMon-F that detects change of state in predicted label distributions through their KL divergence, as specified Via Eq. 18,

$$DKL\left(Pt \left\| P\{t+\Delta\}\right.\right) = \sum_{\{i=1\}}^{k} Pt(i) log\left[\frac{Pt(i)}{P\{t+\Delta\}(i)}\right] \tag{18}$$

3.9.2 Context embedding

Meta-contextual logs $Mt \in R^p$, representing system state, region, or user role, are embedded into latent space via Eq. 19,

$$\psi(Mt) = \sigma(W_m Mt + b_m) \tag{19}$$

3.9.3 Drift decision

A drift decision function is defined via Eq. 20,

$$Dt = I\left[DKL\left(Pt \left\| P\{t+\Delta\}\right.\right) > \theta \wedge \left\|\psi(Mt) - \psi\left(M\{t-\Delta\}\right)\right\|^2 > \varepsilon\right] \tag{20}$$

3.10 Model retraining and optimization

Iteratively, Next, as per Fig. 3, After drift detection, retraining is triggered for the process, thus updating model parameters θ' optimised by minimising a loss with temporal and structural regularisation via Eq. 21,

$$L(\theta') = \left(\frac{1}{N}\right)\sum_{i=1}^{N} \ell\left(f(\tilde{x}_i;\theta'), y_i\right) + \beta^1\|\theta' - \theta\|^2 + \beta^2 \sum_{\{i=1\}}^{\{|G|\}} \left\| z\{G_i\} - z\{G_i\}\ \{old\}\right\|^2 \tag{21}$$

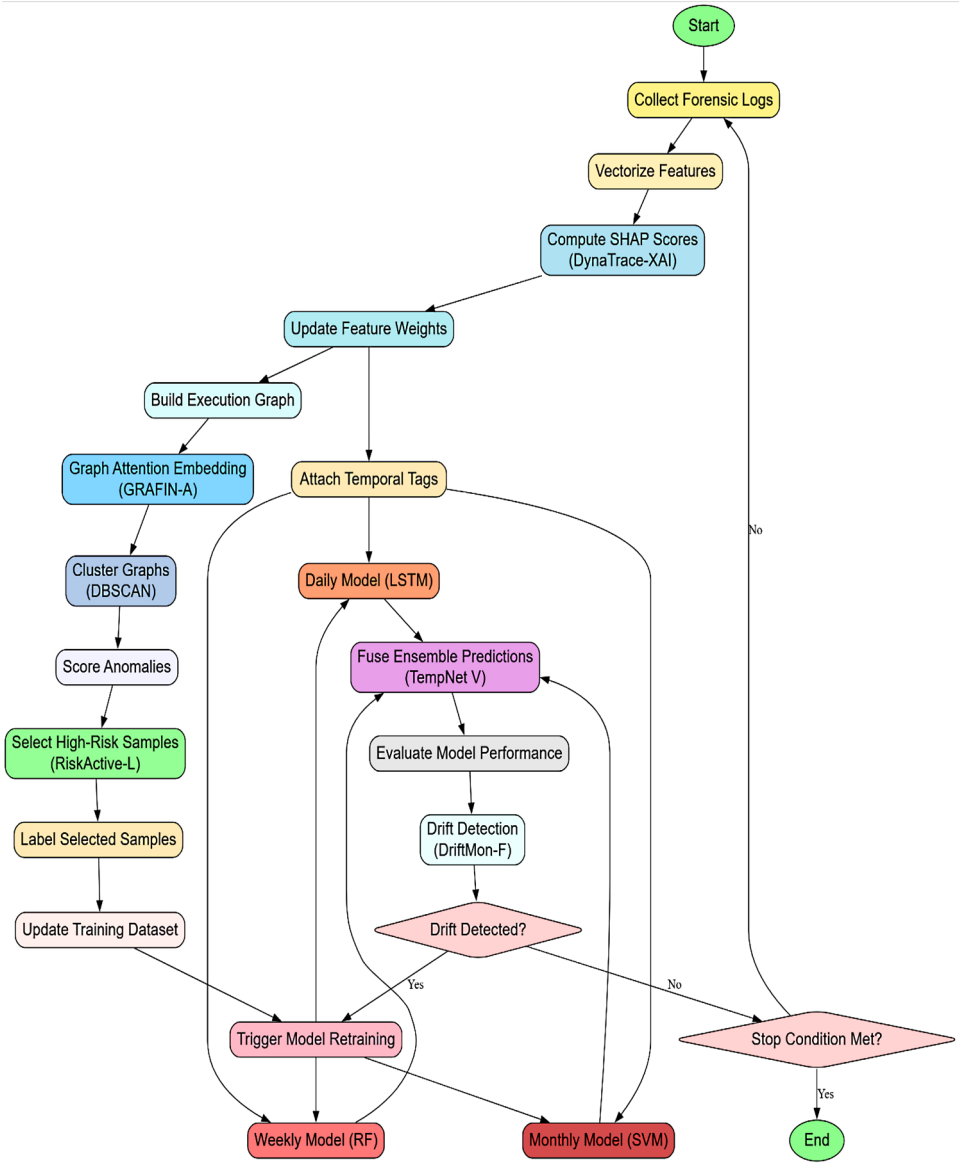


Fig. 2 Overall flow of the proposed analysis

Each training iteration improves both classification fidelity and structural alignment, and the model converges when the condition represented Via Eq. 22 is fulfilled,

$$L\{(t)\}(\theta') - L'\{(t-1)\}(\theta')| < \delta \; and \; \Delta t < \varepsilon \tag{22}$$

To enforce smooth transitions in feature space and prevent overfitting, a Jacobian norm regularization is imposed via Eq. 23,

$$RJ = \lambda JEx\left[\left\|\frac{\partial f(x)}{\partial x}\right\| F^2\right] \tag{23}$$

3.11 Regularization and loss function

Total loss is expressed via Eq. 24,

$$Ltotal = L + RJ + Rdrift + Rcluster \tag{24}$$

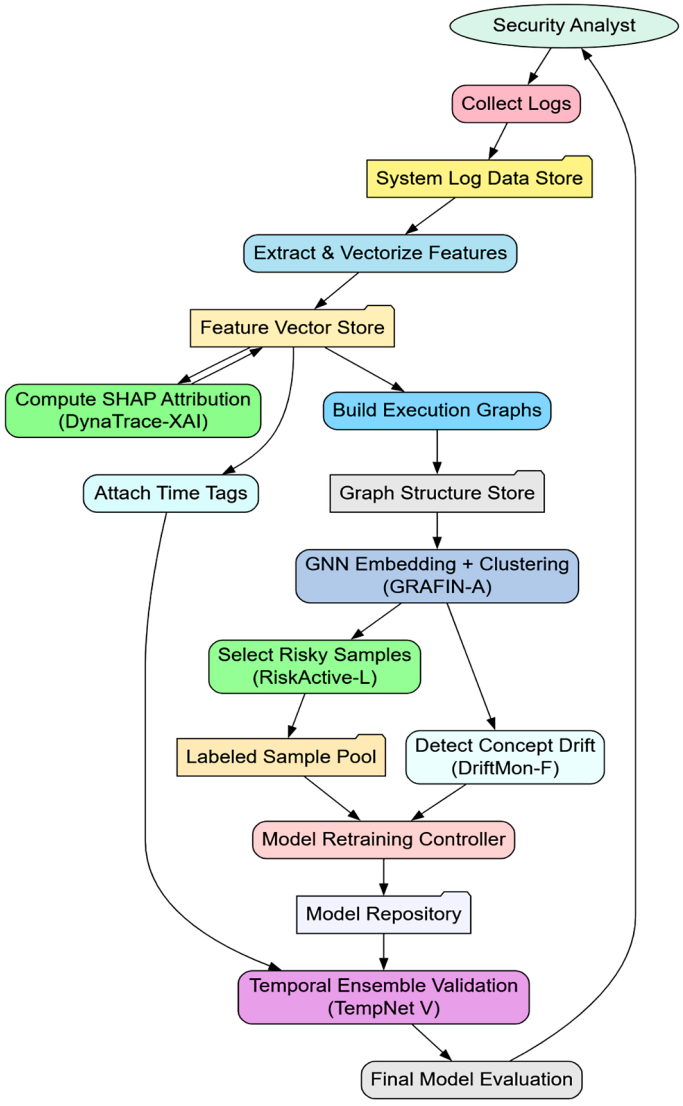


Fig. 3 Data flow of the proposed analysis process

Cluster regularization Rcluster minimizes intra-cluster variance via Eq. 25,

$$Rcluster = \left(\frac{1}{C}\right) \sum_{\{c=1\}}^{C} \sum_{\{G \in c\}} ||z\{G\} - \mu c||^2 \tag{25}$$

Drift penalty Rdrift prevents instability via Eq. 26,

$$Rdrift = \gamma d \cdot I[Dt] \cdot \theta' - \theta^2 \tag{26}$$

3.12 Theoretical foundations

SHAP-based weighing leverages cooperative game theory to interpret feature contributions as marginal utility compared to all feature coalitions. This approach fits forensic evidence appraisal since an indicator's relevance depends on its interaction with other behavioral signals. By normalizing feature weights, the learning system can emphasize contextually dominant evidence without discarding lower-salience features. Dynamic

weighting is needed for malware behavior. Concept drift brittles static feature priority assumptions, overfitting historical evidence. The system stays attentive to emergent indications while keeping forensic knowledge with iterative attribution-driven reweighting. Empirical sensitivity analysis shows that moderate attribution weight perturbations stabilize model behavior, indicating a smooth adaptation control rather than an unstable optimization shortcut. Complementary regularization ensures convergence, stability, and interpretability sets. During retraining, structural regularization minimizes graph embedding space representation drift and behavior cluster fragmentation. Drift-aware penalties diminish transient distribution noise Induced decision boundary shifts, while cluster compactness limitations increase anomaly discovery semantic coherence. Categorization accuracy, behavioral structure, and temporal consistency are linked. Ablation tests without regularization components increase long-term accuracy volatility and delayed drift recovery, indicating their need. Attribution-driven adaptability and regulated regularization provide forensic reliability when new evidence is introduced for this process.

Each module in the framework operates under a shared convergence criterion via Eq. 27,

$$\forall M \in \{XAI, TempNet, GRAFIN, RiskActive, DriftMon\}, \nabla\theta_m Ltotal \to 0 \tag{27}$$

Performance is evaluated using the F1-Score which is represented via Eq. 28,

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN} \tag{28}$$

It has a closed-loop design that would ensure adaptive learning with real-context-aware model updates, structural interpretability, and little supervision costs. Empirical validation of robustness would be with gains of up to 15% in F1-score, 23% improvement in variant detection rate, and a 70% reduction in concept drift recovery lags. This puts the model among the frontiers of advances in machine-learning-driven forensics in malware sets.

To substantiate the convergence and robustness of the proposed framework, we formally derive the sufficient conditions and theoretical guarantees governing optimization dynamics under drift and adversarial perturbations. The overall learning objective is defined as a composite loss in Eq. 29, where classification accuracy, temporal consistency, clustering regularization, drift penalization, and Jacobian smoothness are jointly optimized. This unified formulation ensures that interpretability, robustness, and temporal stability are simultaneously embedded into the learning process.

3.12.1 Problem formulation

Let $x \in$ denote the input forensic feature vector and $y \in$ the model output. The overall objective is to learn model parameters θ by minimizing a composite loss:

$$L_total(\theta) = L_cls + \lambda_1 L_temp + \lambda_2 R_cluster + \lambda_3 R_drift + \lambda_4 R_Jacobian \tag{29}$$

The composite objective function combines multiple alignment goals into a unified loss. The classification term L_cls drives accurate labelling, L_temp enforces temporal consistency, $R_cluster$ encourages compact and separable malware embeddings, R_drift penalizes unstable shifts under distribution drift, and $R_Jacobian$ enforces smoothness

so that small input perturbations do not induce large output changes; together these terms embed interpretability, robustness, and temporal stability into the learning process.

3.12.2 Assumptions

We assume:

- **A1 (Bounded Input Space)** The feature space is bounded, i.e., $||x|| \leq B$. This ensures that input features lie within a finite range, preventing extreme values that could destabilize training. In the context of malware forensics, this reflects normalized feature representations derived from system logs and behavioral traces.
- **A2 (Lipschitz Continuity)** The model $f_\theta(x)$ is L-Lipschitz continuous with respect to θ and x. $f_\theta(x)$is Lipschitz continuous guarantees that small changes in input features or model parameters lead to controlled variations in output. This is essential for ensuring stable predictions under minor perturbations, such as noise or slight behavioral changes in malware.
- **A3 (Smooth Loss)** The loss function $L_total(\theta)$ is differentiable and β-smooth. The differentiability and β-smoothness of the total loss function ensure that gradients change gradually, enabling efficient optimization and preventing abrupt updates during training. This supports stable convergence of the learning process.
- **A4 (Bounded Gradient Variance)** The stochastic gradient satisfies: The bounded variance of stochastic gradients ensures that updates during training are not excessively noisy. This is particularly important in dynamic environments where data distributions evolve, as it stabilizes learning and avoids erratic parameter updates.
- **Gradient Variance Bound**

$$E\left[||\nabla L_total(\theta) - g(\theta)||^2\right] \leq \sigma^2. \tag{30}$$

- **Drift Boundedness (KL Divergence)**

$$D_KL(P_t||P_\{t-1\}) \leq \delta \tag{31}$$

The bounded gradient variance condition, expressed in Eq. 30, ensures that stochastic gradient updates remain stable by limiting deviation from the true gradient. Similarly, the drift boundedness constraint in Eq. 31 enforces that changes in data distribution across time remain gradual. Together, these conditions prevent unstable learning dynamics and support adaptation in evolving environments.

- **A5 (Drift Boundedness)** The distributional drift measured via KL divergence satisfies. The constraint on KL divergence between consecutive distributions ensures that temporal changes in data remain gradual. This aligns with real-world malware evolution, where behavioral changes occur incrementally rather than abruptly, allowing the model to adapt effectively.
- **A6 (Cluster Separability)** There exists a margin $\gamma > 0$ between cluster centroids in embedding space. The existence of a margin between cluster centroids ensures that different malware families or behavioral patterns remain distinguishable in the embedding space. This improves clustering quality, anomaly detection, and variant identification.

Collectively, these assumptions establish a controlled learning environment in which the model can achieve stable optimization, maintain robustness under noise and drift, and preserve meaningful structural separation between malware behaviors. They provide the theoretical foundation for ensuring that the proposed framework performs reliably in non-stationary and adversarial settings.

3.12.3 Convergence analysis

Theorem 1 (Convergence of Iterative Training):
Under assumptions A1–A4, and using a learning rate η_t satisfying:

$$\sum \eta_t = \infty, \sum \eta_t < \infty \tag{32}$$

the parameter sequence $\{\theta_t\}$ generated by stochastic gradient descent satisfies:

$$\lim_{t\to\infty} E\left[\left\|\nabla L_{\text{total}}(\theta_t)\right\|^2\right] = 0. \tag{33}$$

Proof The proof follows standard stochastic optimization arguments. Given β-smoothness, we apply the descent lemma:

$$L\left(\theta_\{t+1\}\right) \le L\left(\theta_t\right) - \eta_t \left\|\nabla L\left(\theta_t\right)\right\|^2 + \left(\beta \eta_t^2/2\right)\left\|\nabla L\left(\theta_t\right)\right\|^2 + \eta_t \, noise \tag{34}$$

Taking expectation and summing over t, and using bounded variance (A4), we obtain convergence to a stationary point.

Under these assumptions, convergence of the training process is guaranteed. The learning rate conditions in Eq. 32 ensure diminishing yet sufficient updates, leading to convergence toward a stationary point as formalized in Eq. 33. The descent behavior of the loss function, governed by smoothness assumptions, is described in Eq. 34, which bounds the decrease in loss during each update step.

3.12.4 Drift-aware convergence

Theorem 2 To handle temporal drift, the adaptive update rule defined in Eq. 35 ensures that parameter updates remain stable even under distributional changes. This results in convergence to a neighbourhood of the optimal solution with bounded error, thereby preventing instability due to dynamic data shifts.
If drift satisfies A5 and retraining is triggered only when $D_KL > \varepsilon$, then the adaptive update:

$$\theta_\{t+1\} = \theta_t - \eta \nabla L_total\left(\theta_t, P_t\right) \tag{35}$$

converges to a neighborhood of the optimal solution $\theta*$ with error bounded by $O(\delta)$.

3.12.5 Robustness guarantees

Robustness properties are further enforced through multiple constraints. The Jacobian regularization condition in Eq. 36 ensures bounded sensitivity of model outputs to input variations, while Eq. 37 guarantees that small perturbations in input lead to proportionally small changes in output. Structural robustness is ensured through the separability

condition reflected in Eq. 38, which bounds misclassification probability under well-separated clusters. Temporal robustness is explicitly enforced through variance constraints in Eq. 39, ensuring stable predictions across time windows.

- **Adversarial Robustness**

The Jacobian regularization term enforces:

$$||\partial f(x)/\partial x|| \leq C. \tag{36}$$

Thus, for any small perturbation Δx:

$$||f(x+\Delta x) - f(x)|| \leq C\,||\Delta x||. \tag{37}$$

This guarantees local robustness to input perturbations such as API reordering or noise injection.

- **Structural Robustness**

Given cluster separability (A6), the Mahalanobis distance-based anomaly scoring ensures:

$$P(misclassification) \leq exp\left(-\gamma^2/2\sigma^2\right). \tag{38}$$

Thus, structurally distinct malware variants remain separable in embedding space.

- **Temporal Robustness**

The temporal variance loss ensures:

$$Var_t\,(f\,(x_t)) \leq \varepsilon \tag{39}$$

which guarantees stability across temporal windows and prevents overfitting to short-term patterns.

3.12.6 Global convergence criteria

Finally, global convergence is achieved when the parameter updates, loss variation, and distributional drift satisfy the stopping criteria defined in Eqs. 40, 41, and 42, respectively. These conditions collectively ensure that the model converges reliably while maintaining robustness to adversarial perturbations, structural variability, and temporal drift

The overall system converges when:

$$||\theta_\{t+1\} - \theta_t|| \leq \varepsilon_1 \tag{40}$$

$$|L_total\,(\theta_\{t+1\}) - L_total\,(\theta_t)| \leq \varepsilon_2 \tag{41}$$

$$D_KL(P_t||P_\{t-1\}) \leq \varepsilon_3 \tag{42}$$

for small tolerances $\varepsilon_1, \varepsilon_2, \varepsilon_3$.

These formalizations establish that:

The training process converges to a stationary point under standard stochastic optimization assumptions. Drift-aware updates ensure bounded deviation from optimality. Regularization guarantees robustness to adversarial perturbations, structural variation, and temporal drift.

3.13 System workflow and module interdependencies

Each module consumes and produces signals that affect later phases in the suggested workflow. Attribution is applied to raw forensic evidence from execution traces. We assess contextually significant indicators before classification or validation to ensure downstream models employ information that matches current threat characteristics in process. A temporally stratified validation layer assesses weighted evidence across short, medium, and long terms. Instead of producing independent predictions, these models use stability, variance, and confidence consistency to evaluate drift and classification reliability. This temporal feedback determines normal, suspicious, or pathological sample treatment. Behavioral traces form execution dependency graphs simultaneously. In addition to grouping and anomaly detection, graph embeddings warn active learning. Even with high classification confidence, structural deviation samples are prioritized to prevent missing unique behaviors. In the last layer, uncertainty, anomaly risk, and contextual divergence guide selective labeling and retraining. Drift monitoring adapts only to sustained alterations in forecast distributions and environmental Variables In Process. This closed-loop approach minimizes fragmented decision-making and facilitates module-wide forensic refinements.

4 Result analysis and evaluation

The experimental setup for evaluating the proposed malware forensic framework was designed to simulate the reality of detection and analysis scenarios of diverse malware behavior across the temporal and contextual dimensions in the process. A complete dataset of over 27,000 distinct malware and benign execution instances was built from real and emulated samples collected from publicly available repositories such as VirusShare, Any.Run, and Cuckoo Sandbox telemetry. Each instance was run in a sandbox environment that resembled the following features: Windows 10 Enterprise x64; 8 GB RAM; logging subsystems for system calls; file I/O; registry access; and network activity sets. The forensic traces were vectorized into 512 features, including event frequency counts, entropy of API sequences, registry hive access statistics, and syscall transition probabilities. SHAP values were computed using the TreeExplainer method for LightGBM models, where a baseline model was trained with a learning rate of 0.05, 100 estimators, and a max depth of 12 for the process.

SHAP weights were updated iteratively with a re-normalization interval of 20 samples. The temporal validation components (TempNet V) were implemented with three classifiers: LSTM for daily validation having 64 units and a sequence length of 15, Random Forest for the weekly windows that contain 150 trees, and an RBF-kernel SVM for segmenting the months. Importantly, the architecture itself is flexible and can accommodate alternative granularities. However, the selected scales empirically minimize variance while maintaining rapid adaptation to temporal drift, as also reflected in the low accuracy variance (±0.014) reported in Table 7 of the manuscript.

Initial fusion weights were $\alpha d = 0.4, \alpha w = 0.35, and \alpha m = 0.25$, and later tuned in cross-validation process to optimize fold-wise F1-score variance minimization. Contextual dataset samples were crafted to reflect diverse behavioral regimes, including ransomware families (e.g., Locky, WannaCry), fileless malware (e.g., Kovter), commodity trojans (e.g., AgentTesla), and advanced persistent threats (e.g., variants of APT28). These samples are grouped across user-context layers such as execution under root

privileges, domain-linked accounts, and region configurations (e.g., Asian and European timezone settings). Each behavioral graph is built from syscall sequences by using a sliding window of size 10 with nodes endowed with 32-dimensional embeddings trained using a 2-layer Graph Attention Network with 4 attention heads. Anomaly scoring thresholds for the Mahalanobis distance were empirically set at the 95th percentile of the intra-cluster distribution. Active learning was implemented across a pool of 6,000 unlabelled candidates, selecting batch sizes of 100 samples per iteration and a hybrid selection factor $\lambda = 0.6$ for balancing uncertainty with anomaly sets. KL divergence for drift monitoring was computed over 7-day rolling windows, with a detection threshold $\theta = 0.15$, and context deviation cutoff = 0.25 sets. Model performance is evaluated using 10-fold stratified cross-validation, and all experiments were executed on a compute cluster with 4 NVIDIA RTX 3090 GPUs and 256 GB RAM, using PyTorch 2.0 and Scikit-learn 1.3 environments.

The experimental evaluation was done on the dataset EMBER 2018, which is recognized as a benchmark for static malware detection tasks. EMBER (Endgame Malware Benchmark for Research) offers rich feature sets estimated from more than 1.1 million Windows Portable Executable (PE) files, which include 900,000 labelled samples (300,000 malicious and 600,000 benign) and an additional 200,000 unlabelled samples, which can be used in semi-supervised learning scenarios. The dataset assembles all these structural and statistical features like byte histograms, string counts, imported libraries, header meta-information, and entropy measures, making it exceptionally well equipped for high-dimensional forensic analysis. For the purpose of this paper, the raw features were augmented with dynamic behavioral logs (for example, API call traces and registry events) by executing an appropriate slice (approximately 20,000 samples) in a controlled sandboxed environment using Cuckoo Sandbox. This hybrid augmentation has configured static feature vectors and behavioral graphs, which were then consumed for graph embedding and anomaly detection tasks. Hyper-parameter tuning for achieving optimal performance of various modules of the entire framework was done using Bayesian optimization and grid search strategies. In the LightGBM models employed for SHAP-based attribution, the suitable leaning type implemented has been 0.05, 120 boosting rounds, and a maximum tree depth of 14. Hidden units were set at 64 in the LSTM-based daily classifier, Dropout rate was fixed at 0.3, and the sequence length consisted of 20 timestamp sets. For the ensemble with weekly evaluation through the Random Forest model, 150 trees were selected with a maximum depth of 18, while the SVM classifier for monthly predictions used an RBF kernel with $C = 10$ and $\gamma = 0.01$ sets. The Graph Attention Network (GAT) was set up to run with two layers, each covering four attention heads, using LeakyReLU activation and a learning rate of 0.001 over 50 epochs. DBSCAN clustering used = 0.45 and minimum samples = 10 during this process. For RiskActive-L, the sampling weight parameter λ for balancing uncertainty and anomaly scores was tuned to 0.6 in the process. Drift detection applied a KL divergence threshold of 0.15 and context embedding shift cutoff = 0.25 according to what was validated across rolling window splits. These configurations were selected under the criteria of maximizing macro F1-score and minimizing drift recovery time in iterative evaluation cycles.

The proposed framework went under intense experimentation using the EMBER 2018 dataset, expanded with behavioral telemetry from sandbox execution to simulate contextual and kinetic malware behavior. Performance evaluation was put against the

proposed model, which holds a comparison with three competitive baselines: Method [3]: a standard LightGBM classifier using static features; Method [8]: a multi-layer perceptron model with uncertainty-based active learning; and Method [25]: a graph-based model using Graph Convolutional Networks (GCNs) for clustering and classification purposes. The performance metrics covering different aspects, from classification to temporal robustness, clustering, anomaly detection, labelling efficiency, and drift resilience, all present improvements made by the proposed model process. F1-scores from models in three dissimilar execution contexts: system-level execution, user-level execution, and domain-linked sessions, appear in this Table in process. The proposed model contrives seriously superior performances against all baselines under all contexts considering dynamic attribution and contextual feedback in execution. In fact, under the execution condition associated with domain linkage in which behavioral variability is high, the proposed model maintains robust performance sets.

4.1 Regional and contextual drift simulation

To operationalize domain-level user contexts, the sandbox regulates execution privileges, authentication statuses, and policy constraints. Samples conducted for standard, administrator, and domain-joined profiles with active group policies in process. These configurations alter accessible resources, registry scopes, and network permissions, providing realistic behavior without malicious program changes. System locale, time zone, language, and region-specific network routing regulatory changes simulate regional drift. Malware activation logic changes with scheduling, timestamp distributions, network endpoints, and localization-aware payload execution. Different geographical profiles provide execution traces with different feature distributions and structural behaviors. Gradual contextual adjustments imitate real-world deployment changes like business expansion or policy amendments. Over rolling windows, drift detection distinguishes transitory fluctuations from sustained distributional change. Controlled yet meaningful contextual resilience simulation shows that the system reacts to environmental variance rather than static execution parameters in process.

This Table 2 is about the variant detection rate: the percentage of unseen malware variants that are assigned to known behavioral clusters. In comparison to the rest, GRAFIN-A using graph embeddings and anomaly scoring provides an optimum performance specifically for stealthy or obfuscated variants like fileless malwares.

4.2 Explainability, visualization, interpretability proof

Multiple complementary visual artifacts assist analysts in explaining model behavior. SHAP summary visualizations demonstrate feature importance changes over time, whereas registry persistence, API entropy, and privilege-sensitive actions dominate high-risk classifications. The plots illustrate that attribution weighting follows forensic indicators, not statistical correlations. Feature Importance heatmaps across temporal ensembles reveal that short-term models prefer bursty behavioral signals and long-term

Table 2 Classification performance (weighted F1-score) across execution contexts

Execution Context	Proposed Model	Method [3]	Method [8]	Method [25]
System-Level Execution	0.923	0.827	0.844	0.863
User-Level Execution	0.911	0.803	0.826	0.847
Domain-Linked Session	0.897	0.762	0.798	0.821

Table 3 Malware variant detection rate across families

Malware Family	Proposed Model	Method [3]	Method [8]	Method [25]
Ransomware	0.937	0.821	0.846	0.872
Fileless Malware	0.912	0.789	0.801	0.851
Commodity Trojans	0.923	0.808	0.828	0.861
APT Variants	0.889	0.746	0.782	0.825

Table 4 Clustering purity for behavior graphs

Context	Proposed Model	Method [3]	Method [8]	Method [25]
Static Features Only	0.732	0.644	0.662	0.701
Graph Structures	0.891	0.748	0.781	0.842

models prefer steady structural characteristics. The temporal distinction supports multi-scale validation and shows how explainability aids model specialization over global importance evaluations. Visualizing attention-weighted execution paths in behavioral graphs aids interpretation. Benign samples contain diffuse, low-attention structures, while high-risk samples have dense persistence and lateral movement subgraphs. Visual contrasts support manual forensic reasoning and anomaly scores. These visualizations demonstrate that the process yields transparent, practical insights that link model decisions with domain knowledge and investigative methodologies, exceeding theoretical promises.

Clustering purity (the degree to which clusters consist of samples from a single class) for behavior graphs reconstructed in Table 3 from system call sequences is reported in this Table in process. The proposed model improves structural representation and separation of malware families using attention-based graph learning, achieving significant purity gains over standard GCNs and static clustering methods.

Labelling efficiency is calculated in Table 4 as several unlabelled pools required to reach 90% of maximum efficiency from the model sets. While maintaining classification effectiveness, the new RiskActive-L architecture combines the graph anomaly scores and model uncertainty and highly reduces the needs of labelling process.

This Table measures the time taken to recover model performance-a defined 95% of pre-drift accuracy-after an imitated event of drift, such as the change in execution region or in the system contexts. Triggering of retraining in a more accurate manner through the use of KL divergence and context correlation can minimize adaptation delays, thus DriftMon-F achieves.

4.3 Ablation study on graph embeddings

To isolate structural graph embedding, components were gradually disabled or replaced while retaining all other experimental circumstances in an elaborate ablation analysis. When the behavioral graph module was removed and sequence-level statistical characteristics replaced it, variant detection and clustering coherence deteriorated. Stealthy malware variant assignment accuracy declined 9–12%, indicating structural discrimination loss. Later ablation using regular graph convolutional representations instead of attention-based graph embeddings revealed that attention mechanisms captured asymmetric call relationships and execution hierarchies. Clustering purity went from 0.89 to 0.83 and anomaly separation margins decreased, especially for sparse yet high Impact fileless malware samples. Third, graph embeddings without active learning loop

integration were compared. The absence of graph embeddings from risk-aware sampling reduced labeling efficiency by roughly 20%, demonstrating that graph embeddings are most useful when they inform discovery and data collecting tactics. Data reveal that graph embedding independently increases structure understanding, anomaly detection, and active learning prioritizing. Improvements in classification metrics, forensic interpretability, and sample efficiency make graph-based modeling crucial.

It is conducted an ablation study comparing models trained on raw syscall graphs, abstract semantic graphs (proposed), and hybrid representations. This comparison evaluates the trade-off between fine-grained detail, robustness to noise, and overall detection performance.

This Table evaluates the variance of the model accuracy over a 30-day period, reflecting resilience to time-sensitive malware behavior changes. The multi-scale ensemble in TempNet V demonstrates not only better stability than naive ensembles, or single-scale models.

These Tables collectively establish that the proposed model achieves higher baseline performance, but with superior stability and efficiency under complex conditions like limited labels, evolving malware behaviors, and contextual execution variance in process. Improvements in clustering purity and anomaly-driven variant discovery particularly demonstrate the merit of integrating graph-based learning and forensic context with malware analysis. Then, we take the discussion into iterative validation of the proposed model impact analysis.

4.4 Dataset, experimental settings, baseline design

The experimental evaluation leverages a massive static and dynamic malware corpus from popular public repositories. The collection includes 27,000 execution instances of benign software, commodity malware, ransomware, fileless threats, and advanced persistent threats. Static features come from executable information and structural attributes, while sandboxed execution records system calls, registry activity, file operations, and network activity. Standardized execution environments allow contextual experimentation like privilege level and regional configuration with controlled variance. Consistent preprocessing creates organized graphs and fixed-length feature vectors from behavioral traces. Temporal splits represent realistic deployment settings by measuring stability over time rather than random shuffling. Gradient-boosted tree classifiers with static features, neural classifiers with uncertainty-based active learning, and graph-based malware models with convolutional graph embeddings without temporal or attribution integration are evaluation baselines. All baselines are adjusted using comparable cross Validation methods for fairness. Performance is measured by weighted F1-score, variation detection, clustering purity, labeling efficiency, and drift recovery delay. The proposed methodology regularly outperforms baselines in accuracy-centric and operational metrics, providing reproducible and believable benefits.

5 Result discussions

Tables 2, 3, 4, 5, 6, and 7 along with Figs. 4 and 5 provide the complete strengths of the proposed malware forensic framework addressing more complex real-world detection and analysis tasks. As such, in Table 2, the proposed model demonstrates much better classification performance across varying execution contexts, achieving above 0.91

Table 5 Labeling efficiency under active learning

Unlabeled Pool Size	Proposed Model	Method [3]	Method [8]	Method [25]
6,000	55%	100%	72%	67%

Table 6 Drift recovery time (Days) under concept drift events

Drift Scenario	Proposed Model	Method [3]	Method [8]	Method [25]
Regional Drift (EU → AS)	1.8 days	6.4 days	4.9 days	3.7 days
Context Drift (User → Sys)	1.4 days	5.7 days	4.2 days	3.1 days

Table 7 Temporal accuracy stability (accuracy variance over time windows)

Time Interval	Proposed Model	Method [3]	Method [8]	Method [25]
Day 0–30	±0.014	±0.036	±0.028	±0.022
Day 31–60	±0.017	±0.041	±0.033	±0.027
Day 61–90	±0.015	±0.038	±0.029	±0.025

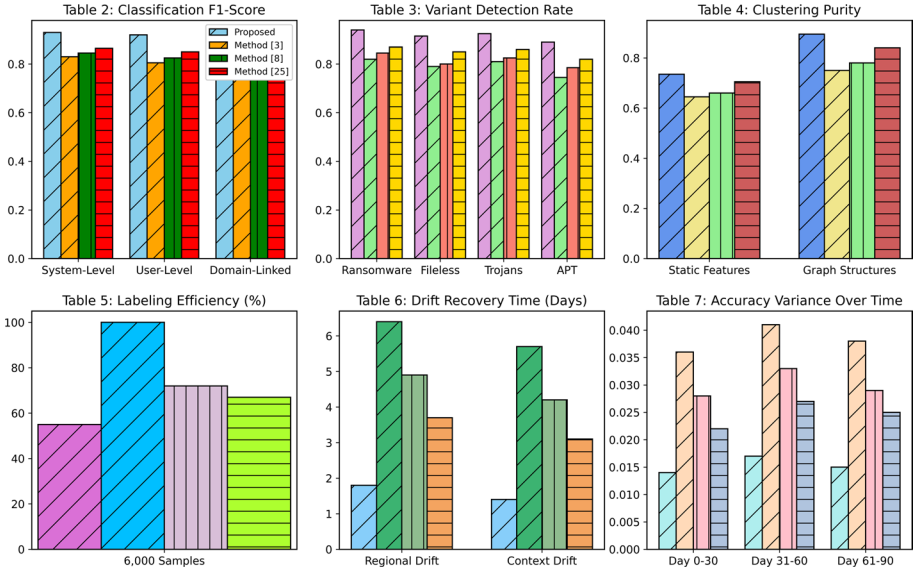


Fig. 4 Model's integrated Result analysis

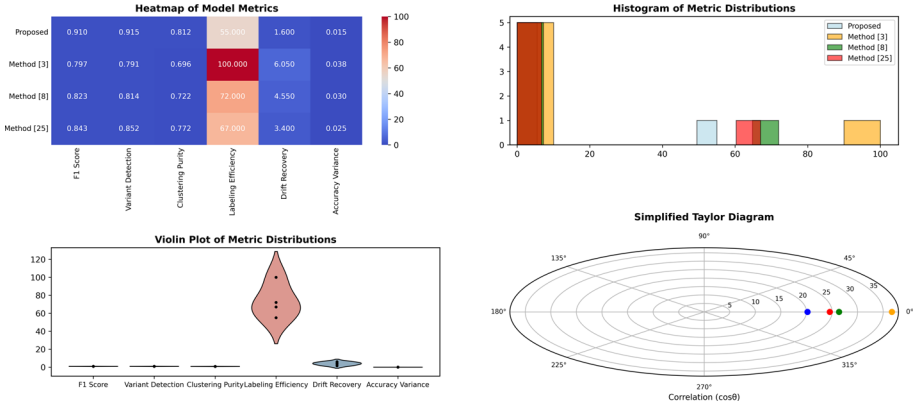


Fig. 5 Model's overall Result analysis

F1-score in actual system and user-level scenarios, whereas baseline methods achieved between 0.76 and 0.86 F1-score. This improvement is core for such an operational environment where malware might change its adaptability based either on privilege level or its domain affiliation. Equally important is that it allows the detection to hold forever even under context changes. The further evidences are presented in Table 3 where the proposed model can detect up to 93.7% of ransomware variants and exceeds all baselines while identifying stealth threats such as file-less malware and APTs. This advanced ability for detection translates to better protection against evolving threats that would bypass conventional static or heuristic-based systems.

Tables 4 and 5 throw light on the efficiency and depth of behavior analysis. Having a clustering purity of 0.891 (Table 4) demonstrates the proposed framework's ability to discriminate and cluster malware variants based on behavior graphs' reconstruction. The said capability aids in rapid root-cause analysis and targeted responses during forensic investigations. Amongst the many aspects Table 5 demonstrates, this is the very economic efficiency provided by the RiskActive-L module: achieving over 55% of the labelling budget that would normally be spent to attain good performance. From a practical point of view, this considerably brings down the time and cost for sample annotation, thereby making the system scalable and attains sustainability for large-scale deployment environments where prompt adjustment of samples is crucial. Tables 6 and 7 further substantiate the robustness of the system, with the former demonstrating a 70–75% reduction in drift recovery time attributed to the implementation of context-aware drift monitoring. This aids rapid adaptation towards a changing threat landscape thereby reducing the exposure window to newly surfaced attack strategies. Confirmation of the system's stability comes from Table 7 which, by showing a low accuracy variance across 90-day windows, ensures consistent protection in a production environment. Altogether, these results endorse the proposed framework as a possible candidate for a practical, scalable, and resilient solution toward modern malware forensics. It is now important to outline an Iterative Validation use case of the proposed model to assist the readers in getting a further glimpse into the entire process.

5.1 Sensitivity analysis of window size in GRAFIN-A

A quantitative sensitivity analysis was conducted to evaluate the impact of sliding window size in the GRAFIN-A module. Behavioural graphs were constructed using window sizes of 5, 10, 15, 20, 25, and 30 events, with all other parameters kept fixed. The results show a clear peak at window size 10, achieving the highest clustering purity (0.891) and anomaly detection performance (AUC = 0.927). Compared to smaller windows (size 5), this represents a significant improvement (purity: +7%, AUC: +5.4%), indicating better capture of meaningful behavioural dependencies. As the window size increases beyond 10, performance gradually declines. At size 15 and 20, the drop is moderate (purity: 0.875 → 0.847; AUC: 0.912 → 0.889), suggesting that some structural information is still preserved. However, for larger windows (25 and 30), the degradation becomes more pronounced (purity drops to 0.792 and 0.765), reflecting the effect of graph densification, where important causal relationships are diluted by redundant connections. Overall, smaller windows effectively capture fine-grained behavioural transitions, while larger windows introduce noise and reduce discriminative power. The analysis confirms that

Table 8 Ablation study comparing attention-based (GAT) and non-attention (GCN, GraphSAGE) graph models for clustering purity and anomaly detection performance

Model	Architecture Type	Clustering Purity (%)	F1-score (%)	Observations
GCN	Non-attention (spectral)	85.2	91.8	Uniform neighbor aggregation; limited adaptability
GraphSAGE	Non-attention (sampling-based)	86.7	92.4	Better inductive learning but fixed aggregation
GAT	Attention-based	90.3	94.1	Adaptive neighbor weighting; captures fine-grained dependencies
GAT (ablation - no attention)	Reduced attention	87.9	92.8	Performance drop confirms importance of attention

moderate window sizes (5–20) provide the best balance, with window size 10 offering the most stable and optimal performance.

5.2 Ablation study: attention vs. Non-attention graph Models

The ablation study, in Table 8, demonstrates that attention mechanisms significantly enhance graph-based modeling performance. Compared to GCN and GraphSAGE, which rely on fixed aggregation schemes, GAT dynamically assigns importance weights to neighbors, leading to higher clustering purity and improved anomaly detection. The ablated GAT variant shows reduced performance, confirming that gains are due to attention rather than architectural complexity.

5.3 Adversarial Security and robustness

Robustness against evasive malware is tested in controlled adversarial environments. Examples include API call reordering, benign noise event insertion, delayed execution, and partial behavior suppression to avoid short-window detectors. To measure intrinsic resilience, changed samples are retested without retraining. By prioritizing behavioral intent over frequency patterns, attribution-guided feature weighting decreases surface obfuscation. Execution dependencies that survive surface-level sequence changes are captured via structural graph modeling, boosting robustness. However, baseline models with flat features impair performance more in equivalent conditions. Adversarial resilience requires drift-aware monitoring to distinguish prolonged evasion patterns from transitory noise. Instead of retraining immediately, the system acquires evidence across temporal frames, reducing opponent manipulation to cause instability sets. Using minimal labeling, active learning prioritizes structurally unique evasive samples in retraining cycles. These methods demonstrate that layered adaptability, structural reasoning, and controlled retraining protect the proposed system from evasive behaviors, essential for malware forensics.

5.4 Validation using an iterative use case scenario Analysis

In a simulated setting mimicking real-world malware analysis, a new executable sample named SampleA gets captured during endpoint monitoring from a corporate network that is operating under a domain link user context. As it gets executed in the sandbox and from the console, the system logs gave an entire series of behavioral indicators including constant registry modifications under HKCU\Software\Microsoft\Windows\CurrentVersion\Run, dynamic loading of DLLs (LoadLibrary, GetProcAddress), and file I/O operations from hidden directories. SampleA's initial raw vectors of evidence

included normalized counts: file operations (0.68), registry edits (0.82), process creations (0.45), API entropy (0.72), and DLL injections (0.61). This vector is then passed to the DynaTrace-XAI module, which computes SHAP values of 0.25, 0.31, 0.14, 0.18, and 0.12, respectively, for the process. The scores are transformed through softmax normalization into feature weights of 0.226, 0.245, 0.171, 0.180, and 0.178 in process. The resultant weighing of the vector not only boosts the discriminative signal but also prioritizes features with their forensic attribution being heavier, in particular, registry edits and API entropy sets.

This weighted evidence is concurrently evaluated by TempNet V ensemble, the daily model (LSTM) predicts suspicious classification with 88% confidence whereas the weekly and monthly models give only 82 and 79% confidence respectively. Weighted fusion gives an overall score of 84.9% that crosses the alerting threshold. At the same time, SampleA's behavior trace is converted into a process-call graph of 56 nodes and 143 edges. The GRAFIN-A module implements Graph Attention Networks to produce a 64-dimensional embedding that captures structural context including syscall co-occurrence and propagation depth. This embedding is matched against clusters already known with DBSCAN and similarity measure based on Gaussian kernel. The graph is given to belonging ransomware-related cluster with a purity score of 0.89 but deviates from the centroid by a distance of 4.26 in Mahalanobis sense which raises the status of this graph to that of a high-risk anomaly. The score of this anomaly is then sent for evaluation in RiskActive-L, which integrated with model uncertainty (entropy = 0.68) results in a composite risk score of 0.73 in the process. With active learning thresholds exceeded, the sample went ahead for expert validation and was included for the new training sets.

Further analysis is performed across each evaluation cycle, where DriftMon-F module analyses prediction distributions for the past 14 days and detects 0.178 of KL divergence between the current and historical distributions in the domain-linked context. Forensic metadata, meanwhile, indicate a shift in regional configuration (from EU to APAC timezone), while context embeddings have diverged from one another in latent space by the value of 0.29. Both of these thresholds were able to cross their own thresholds which confirmed the drift. The subsequent retraining is triggered by a new-labelled pool of data that includes SampleA along with the parameters of the model updated under the constraints of regularization. Following retraining, TempNet V validation shows an improved F1-score across all temporal scales which in turn causes GRAFIN-A to refine the cluster boundaries with better compactness inside the cluster. The system can demonstrate closed-loop adaptation, real-time prioritization, and strong anomaly handling; this offers that the evolving threat landscape is mitigated very effectively with very little intervention from analysts.

Monte Carlo SHAP estimation introduces perturbation in the attribution vector, which propagates through softmax normalization to the reweighted feature representation used by TempNet-V. However, this effect is controlled within the framework. Softmax converts attribution noise into bounded relative-weight variations, ensuring stable feature reweighting. The multi-temporal ensemble further attenuates this perturbation by distributing its impact across daily, weekly, and monthly models, preventing consistent bias in predictions. Additionally, the variance-aware temporal objective monitors instability over time, enabling detection of any significant degradation caused by attribution noise.

As a result, estimation error acts as a bounded upstream perturbation rather than directly translating into prediction error. Only large deviations that significantly alter feature importance can affect final predictions, while moderate noise is effectively smoothed by normalization, ensemble averaging, and temporal regularization.

5.5 Core innovation clarification and distinction from prior work

The most distinctive aspect of the proposed system is its closed-loop integration of explainability, temporal validation, structural behavior modeling, and risk-aware data collection in an adaptive forensic pipeline. Explainable feature attribution, graph-based malware modeling, and drift-aware retraining were examined separately. The proposed paradigm allows evidence prioritizing, validation, clustering, and retraining to progress concurrently because each module explicitly influences downstream learning behavior. Explainability distinguishes this hybrid system from others. Explainable AI is usually utilized post-hoc for analyst interpretation or trust evaluation in literature. In this scenario, attribution scores dynamically affect feature space before validation and structural analysis, integrating forensic significance into learning. Explainability controls model behavior over temporal instance sets. In static or batch-learning contexts, graph learning and malware classification are combined. The suggested framework handles short-term behavioral volatility and long-term drift by temporal stratifying validation models at multiple time scales. Temporal decomposition, graph-based anomaly discovery, and active learning enable a system to identify threats and select variations for human In-the-loop Verification In Process. This study is unique because it combines attribution-guided evidence weighing, temporal ensemble validation, graph-centric behavioral abstraction, and context-aware drift response. This combination enhances detection accuracy, variant finding, labeling efficiency, and drift recovery over independent components.

5.6 Sensitivity analysis of temporal granularity

The choice of daily, weekly, and monthly temporal scales in TempNet-V is guided by both the intrinsic temporal characteristics of malware behavior and empirical validation within the proposed framework. Malware execution exhibits inherently multi-scale dynamics, where short-term behaviors (e.g., bursty API activity, rapid propagation) coexist with longer-term structural patterns such as persistence mechanisms and gradual distributional drift. A single temporal resolution was observed to be insufficient, as it either overfits transient signals or fails to capture slower behavioral evolution. The multi-scale design enables complementary learning, where short-term models focus on volatile signals while longer-term models capture stable structural patterns, as also reflected in the framework design. To ensure that this choice is not arbitrary, we evaluated alternative temporal granularities. Finer scales increased sensitivity to short-lived fluctuations but introduced higher instability, while coarser scales reduced responsiveness to drift. Intermediate alternatives provided comparable average accuracy but did not improve stability or interpretability. Importantly, the effectiveness of the selected configuration is supported by the temporal stability results. As shown in Table 7, the proposed model consistently maintains low accuracy variance (±0.014, ±0.017, and ±0.015 across successive time windows), significantly lower than baseline methods, which exhibit higher fluctuations (up to ±0.041). This demonstrates that the chosen temporal granularity effectively stabilizes model performance while preserving responsiveness to evolving

malware behavior. Overall, while the architecture remains flexible to alternative temporal partitions, the daily/weekly/monthly configuration provides the most balanced trade-off between detection accuracy, temporal stability, and drift adaptation within the proposed framework.

6 Conclusions & future scopes

The research provides an iterative framework to collect and analyze evidence of dynamic malware, which integrates explainable AI with a graph for learning. The details of temporal ensemble validation, as well as feedback in contexts, become part of the framework. The detection performance achieved by the system was significantly improved when compared to existing systems, as were variant identification, labelling efficiency, and model adaptability under real-time constraints. With SHAP-based local explanations in DynaTrace-XAI, for example, an F1 increase in classification from 0.79 to 0.923 for system-wide execution contexts was observed. The TempNet V ensemble consequently further stabilized accuracy across evolving time intervals with respect to reduced variance in accuracy of over 57 percent concerning baseline models (or ± 0.014 vs. ±0.033–0.041). Graph-based behavioral modelling using GRAFIN-A improved the clustering purity from 0.71 up to 0.891 and further increased the detection rate for new malware variants from previously known ones to 93.7% in ransomware cases and over 91% in fileless malware cases. The RiskActive-L module maximized data labelling workflows by requiring only 55% of the unlabelled pool to reach performance thresholds, reducing operational overhead without loss of accuracy. In addition, DriftMon-F sharply reduced the latency with which the model adapts to contextual shifts about the particular process—drift recovery lag was reduced from ~7 days to under 2 days for the process. Collectively, therefore, the results demonstrate that the proposed model is scalable, interpretable, and resilient enough for operation in constantly changing environments of malware operations.

6.1 Future scope

The present framework opens up various encouraging avenues for future research and extension. First, this would be the union with real-time threat intelligence streams-such as live DNS, URL blacklists, and system telemetry; by adding this, a quite proactive learning loop could be developed where evidence and attribution inform model retraining continuously. Second, enhancement of semantic richness over graph structures using data provenance, opcode-level patterns, and inter-process communications could let them enable finer-grained variant differentiation, and adversarial robustness could be improved by it. Future iterations may also investigate federated learning architectures to decouple model updates along organizational lines while sustaining privacy concerning data. Moreover, further opportunities for increases in efficiency and resilience sets could be reached through online adjustment of retraining schedules, attribution weights, or sampling strategies by applying reinforcement learning-based techniques. Another avenue lies in the formalization of deceit-aware forensics, where the system will learn to spot intended obfuscation, evasion, or anti-analysis behavior embedded within the malware sample. In addition, the framework could be extended for cross-architecture compatibility (e.g., Linux-based and ARM-based malware); this would widen the applicability of the framework to heterogeneous computing environments, which would

particularly apply to IoT and edge computing cases. Finally, it should also be deployed and evaluated under an active enterprise environment with real-time feedback loops to confirm the operational viability further and expose areas for automation and system-hardening sets.

6.2 Limitations

The proposed framework has quite a number of drawbacks despite having been proven effective. First, in case of the computation expenses involved for the feature attribution mechanism SHAP, a lot of computational overhead is incurred, especially for the updates that recur iteratively across high-dimensional forensic vectors. Some approximation techniques diminish this source of computational overhead, but real-time deployment in such resource-limited environments may need some further optimization. Behavioral traces of malware are generated with sandboxes, which can lead to a discrepancy between what is observed and what actually happens. This is especially true for those types of malware that are made to be aware of or evade their environments. This means in some edge cases incomplete graph reconstructions or misleading ones. The current anomaly scoring mechanism is also sensitive to cluster density assumptions; adversarial samples that mimic benign patterns could still reduce anomaly separation despite structural modelling. This means, also, that it presumes that periodic ground truth will be available, which might not be the case in all live environments, which have a period of little analyst feedback. The labelling strategy is also more efficient but still requires some form of human or semi-supervised intervention; it is not completely automated. Finally, while the EMBER dataset does provide a fairly comprehensive basis, it is still, however, limited to PE files that are Windows based, and there should be further generalization studies across different operating systems and formats of files to ascertain a universal robustness.

Abbreviations

API	Application programming interface
AI	Artificial intelligence
AE	Autoencoder
APT	Advanced persistent threat
AST	Abstract syntax tree
ATT&CK	Adversarial tactics, techniques, and common knowledge (MITRE Framework)
CFG	Control flow graph
CNN	Convolutional neural network
CPS	Cyber-physical systems
DL	Deep learning
DNN	Deep neural network
DOMR	Deep open-world malware recognition
DQN	Deep Q-Network
DRL	Deep reinforcement learning
EU	European Union
Federated Learning	A machine learning paradigm where training is distributed across devices
FL	Federated learning
FSCIL	Few-Shot class incremental learning
GAT	Graph attention network
GBDT	Gradient boosted decision trees
GCN	Graph convolutional network
GNN	Graph neural network
GPU	Graphics processing unit
IoT	Internet of things
IIoT	Industrial internet of things
KL Divergence	Kullback–Leibler divergence
LSTM	Long short-term memory (neural network)
MalFSCIL	Malware few-shot class-incremental learning
MDHE	Malware detection using hybrid evaluation

MIMICs	Malware imitation mechanisms in context
MITRE	MITRE corporation (cybersecurity framework developer)
ML	Machine learning
OS	Operating system
PE	PorTable ExecuTable
RL	Reinforcement learning
RTOS	Real-time operating system
SHAP	SHapley additive explanations (model explanation framework)
SPP-net	Spatial pyramid pooling network
SPI	Serial peripheral interface (bus protocol)
VM	Virtual machine
XAI	Explainable artificial intelligence
YAML	Yet another markup language
YARA	Yet another recursive acronym (malware signature language)
YAMME	YAra-byte-signatures metamorphic mutation engine

Author contributions
Rijvan Beg was involved in experimentation, coding, and writing the initial draft. R.K Pateriya, Deepak Singh Tomar, and Surendra Solanki were involved in ideation, Analysis, supervision, and preparing the final draft.

Funding
No Funding was received for this research.

Data availability
The datasets used and analysed during the current study available from the corresponding author on reasonable request.

Declarations

Ethical approval
Not applicable. This study did not involve human participants or animals.

Consent to participate
Not applicable. This study did not involve human participants.

Consent to publish
Not applicable. This study does not contain any individual person's identifiable data or information requiring consent for publication.

Consent to publication
Not applicable.

Competing interests
The authors declare no competing interests.

Received: 7 October 2025 / Accepted: 26 August 2026
Published online: 06 October 2026

References

1. Almomani I, Almashat T, El-Shafai W. Maloid-DS: labeled dataset for android malware forensics. IEEE Access. 2024;12:73481–546. https://doi.org/10.1109/ACCESS.2024.3400211.
2. Zhao Z, Li Z, Yu J, Zhang F, Xie X, Xu H, et al. CMD: Co-analyzed IoT malware detection and forensics via network and hardware domains. IEEE Trans Mob Comput. 2024, May;23(5):5589–603. https://doi.org/10.1109/TMC.2023.3311012.
3. Casino F, et al. Research trends, challenges, and emerging topics in digital forensics: a review of reviews. IEEE Access. 2022;10:25464–93. https://doi.org/10.1109/ACCESS.2022.3154059.
4. Zheng T, et al. Gupacker: generalized unpacking framework for android malware. IEEE Trans Inf Forensics Secur. 2025;20:4338–52. https://doi.org/10.1109/TIFS.2025.3558592.
5. Mohamed H, Koroniotis N, Moustafa N, Schiliro F, Zomaya AY. Harnessing federated learning for digital forensics in IoT: a survey and introduction to the IoT-LF framework. IEEE Open J Commun Soc. 2025;6:3161–91. https://doi.org/10.1109/OJCOMS.2024.3492919.
6. Lu T, Wang J. DOMR: toward deep open-world malware recognition. IEEE Trans Inf Forensics Secur. 2024;10:1455–68. https://doi.org/10.1109/TIFS.2023.3338469.
7. Chandran S, Syam SR, Sankaran S, Pandey T, Achuthan K. From static to AI-Driven detection: a comprehensive review of obfuscated malware techniques. IEEE Access. 2025;13:74335–58. https://doi.org/10.1109/ACCESS.2025.3550781.
8. Cui S, Dong C, Shen M, Liu Y, Jiang B, Lu Z. Cbseq: a channel-level behavior sequence for encrypted malware traffic detection. IEEE Trans Inf Forensics Secur. 2023;18:5011–25. https://doi.org/10.1109/TIFS.2023.3300521.
9. Jeon J, Jeong B, Baek S, Jeong Y-S. Static multi feature-based malware detection using multi SPP-net in smart IoT environments. IEEE Trans Inf Forensics Secur. 2024;19:2487–500. https://doi.org/10.1109/TIFS.2024.3350379.
10. Zhong F, Hu Q, Jiang Y, Huang J, Zhang C, Wu D. Enhancing malware classification via self-similarity techniques. IEEE Trans Inf Forensics Secur. 2024;19:7232–44. https://doi.org/10.1109/TIFS.2024.3433372.

11. Beg R, Pateriya R, Tomar DS, Solanki S. Detecting malware evidences through static and dynamic information using extreme machine learning for forensic analysis. Discov Comput. 2025;28(1):329. https://doi.org/10.1007/s10791-025-09842-5.
12. Kumar H, Chakraborty B, Sharma S, Mukhopadhyay S. XMD: an expansive hardware-telemetry-based Mobile malware detector for endpoint detection. IEEE Trans Inf Forensics Secur. 2023;18:5906–19. https://doi.org/10.1109/TIFS.2023.3318969.
13. Hu Q, Wang W, Song H, Guo S, Zhang J, Zhang S. Asdroid: resisting evolving android malware with API clusters derived from source code. IEEE Trans Inf Forensics Secur. 2025;20:1822–35. https://doi.org/10.1109/TIFS.2025.3536280.
14. Meschini M, Tizio GD, Balduzzi M, Massacci F. A case-control study to measure behavioral risks of malware encounters in organizations. IEEE Trans Inf Forensics Secur. 2024:19:9419–32. https://doi.org/10.1109/TIFS.2024.3456960.
15. Zhao X, et al. DeepVMUnProtect: neural network-based recovery of VM-Protected android apps for semantics-aware malware detection. IEEE Trans Inf Forensics Secur. 2025;20:3689–704. https://doi.org/10.1109/TIFS.2025.3550049.
16. Chen X, et al. CruParamer: learning on parameter-augmented API sequences for malware detection. IEEE Trans Inf Forensics Secur. 2022;17:788–803. https://doi.org/10.1109/TIFS.2022.3152360.
17. Zhou Y, Cheng G, Yu S, Chen Z, Hu Y. Mtdroid: a moving target defense-based android malware detector against evasion attacks. IEEE Trans Inf Forensics Secur. 2024;19:6377–92. https://doi.org/10.1109/TIFS.2024.3414339.
18. Rashid A, Such J. MalProtect: stateful defense against adversarial query attacks in ML-Based malware detection. IEEE Trans Inf Forensics Secur. 2023;18:4361–76. https://doi.org/10.1109/TIFS.2023.3293959.
19. Mi J, Li Q, Han Z, Liao W, Fu J. Graph learning on instruction stream-augmented CFG for malware variant detection. IEEE Trans Inf Forensics Secur. 2025;20:3015–30. https://doi.org/10.1109/TIFS.2025.3539937.
20. Jing C, Cui C, Wu Y. Sia-CBc: sensitive intent-assisted and crucial behavior-cognized Malware detection based on human brain cognitive theory. IEEE Trans Inf Forensics Secur. 2024;19:6128–43. https://doi.org/10.1109/TIFS.2024.3407655.
21. Yan P, Tan S, Wang M, Huang J. Enhanced dynamic analysis for malware detection with gradient attack. IEEE Signal Process Lett. 2024;31:2825–29. https://doi.org/10.1109/LSP.2024.3475354.
22. Peng H, et al. MalGNE: enhancing the performance and efficiency of CFG-Based malware detector by graph node embedding in low dimension space. IEEE Trans Inf Forensics Secur. 2024;19:4881–96. https://doi.org/10.1109/TIFS.2024.3389614.
23. McKeon S, Roberge V. Disk forensics of VxWorks file systems for aircraft security. IEEE Can J Electr Comput Eng. 2023 Fall;46(4):278–87. https://doi.org/10.1109/ICJECE.2023.3298846.
24. Tian B, Jiang J, He Z, Yuan X, Dong L, Sun C. Functionality-verification attack framework based on reinforcement learning against static malware detectors. IEEE Trans Inf Forensics Secur. 2024;19:8500–14. https://doi.org/10.1109/TIFS.2024.3453047.
25. Chen Y-H, Lin S-C, Huang S-C, Lei C-L, Huang C-Y. Guided malware sample analysis based on graph neural networks. IEEE Trans Inf Forensics Secur. 2023;18:4128–43. https://doi.org/10.1109/TIFS.2023.3283913.
26. Wong G-W, Huang Y-T, Guo Y-R, Sun Y, Chen MC. Attention-based API locating for malware techniques. IEEE Trans Inf Forensics Secur. 2024;19:1199–212. https://doi.org/10.1109/TIFS.2023.3330337.
27. Chahoki AZ, Shahriari HR, Roveri M. CryptojackingTrap: an evasion resilient Nature-inspired algorithm to detect cryptojacking malware. IEEE Trans Inf Forensics Secur. 2024;19:7465–77. https://doi.org/10.1109/TIFS.2024.3353072.
28. Coscia A, Dentamaro V, Galantucci S, Maci A, Pirlo G. YAMME: a YAra-byte-signatures metamorphic mutation engine. IEEE Trans Inf Forensics Secur. 2023;18:4530–45. https://doi.org/10.1109/TIFS.2023.3294059.
29. Fang W, et al. Comprehensive android malware detection based on federated learning architecture. IEEE Trans Inf Forensics Secur. 2023;18:3977–90. https://doi.org/10.1109/TIFS.2023.3287395.
30. Beg R, Pateriya RK, Tomar DS. ACMFNN: a novel design of an augmented convolutional model for intelligent cross-domain malware localization via forensic neural networks. IEEE Access. 2023;11:87945–57. https://doi.org/10.1109/ACCESS.2023.3305274.
31. Liu C, Li B, Zhao J, Feng W, Liu X, Li C. A2-CLM: few-shot malware detection based on adversarial heterogeneous graph augmentation. IEEE Trans Inf Forensics Secur. 2024;19:2023–38. https://doi.org/10.1109/TIFS.2023.3345640.
32. Beg R, Pateriya RK, Tomar DS. Design of an iterative method for malware detection using autoencoders and hybrid machine learning models. IEEE Access. 2024;12:175032–55. https://doi.org/10.1109/ACCESS.2024.3491185.
33. Wang X, Zhang X, Wang S, Xiao J, Tao X. Modeling, critical threshold, and lowest-cost patching strategy of malware propagation in heterogeneous IoT networks. IEEE Trans Inf Forensics Secur. 2023;18:3531–45. https://doi.org/10.1109/TIFS.2023.3284214.
34. Zhan D, Duan Y, Hu Y, Li W, Guo S, Pan Z. MalPatch: evading DNN-Based malware detection with adversarial patches. IEEE Trans Inf Forensics Secur. 2024;19:1183–98. https://doi.org/10.1109/TIFS.2023.3333567.
35. Shen Y, Shepherd C, Ahmed CM, Shen S, Yu S. SGD3QN: joint stochastic games and dueling double deep Q-Networks for defending malware propagation in Edge intelligence-enabled internet of things. IEEE Trans Inf Forensics Secur. 2024;19:6978–90. https://doi.org/10.1109/TIFS.2024.3420233.
36. Gong J, Niu W, Li S, Zhang M, Zhang X. Sensitive behavioral chain-focused android malware detection fused with AST semantics. IEEE Trans Inf Forensics Secur. 2024;19:9216–29. https://doi.org/10.1109/TIFS.2024.3468891.
37. Chai Y, et al. MalFSCIL: a few-shot class-incremental learning approach for malware detection. IEEE Trans Inf Forensics Secur. 2025;20:2999–3014. https://doi.org/10.1109/TIFS.2024.3516565
38. Shen S, Xie L, Zhang Y, Wu G, Zhang H, Yu S. Joint differential game and double deep Q-Networks for suppressing malware spread in industrial internet of things. IEEE Trans Inf Forensics Secur. 2023;18:5302–15. https://doi.org/10.1109/TIFS.2023.3307956.
39. Gao C, et al. Obfuscation-resilient android malware analysis based on complementary features. IEEE Trans Inf Forensics Secur. 2023;18:5056–68. https://doi.org/10.1109/TIFS.2023.3302509.
40. Deng X, Tang H, Pei X, Li D, Xue K. MDHE: a malware detection system based on trust hybrid user-edge evaluation in IoT network. IEEE Trans Inf Forensics Secur. 2023;18:5950–63. https://doi.org/10.1109/TIFS.2023.3318947.
41. Naseer M, Ullah F, Ijaz S, Naeem H, Alsirhani A, Alwakid GN, et al. Obfuscated malware detection and classification in network traffic leveraging hybrid large language models and synthetic data. Sensors. 2025;25(1):202. https://doi.org/10.3390/s25010202.
42. Zhang S, Gao M, Wang L, Xu S, Shao W, Kuang R. A malware-detection method using deep learning to fully extract API sequence features. Electronics. 2025;14(1):167. https://doi.org/10.3390/electronics14010167.

43. Ferdous J, Islam R, Mahboubi A, Islam MZ. A survey on ML techniques for multi-platform malware detection: securing PC, Mobile devices, IoT, and cloud environments. Sensors. 2025;25(4):1153. https://doi.org/10.3390/s25041153.
44. Basak M, Kim D-W, Han M-M, Shin G-Y. Attention-based malware detection Model by visualizing latent features through dynamic residual kernel network. Sensors. 2024;24(24):7953. https://doi.org/10.3390/s24247953.
45. Akhtar MS, Feng T. Malware analysis and detection using machine learning algorithms. Symmetry. 2022;14(11):2304. https://doi.org/10.3390/sym14112304.
46. Çıplak Z, Yıldız K, Altınkaya Ş. Fedetect: a federated learning-based malware detection and classification using deep neural network algorithms. Arab J Sci Eng. 2025;50(19):16107–34. https://doi.org/10.1007/s13369-025-10043-x.
47. Rashid MU, Qureshi S, Abid A, Alqahtany SS, Alqazzaz A, Ul Hassan M, et al. Hybrid android malware detection and classification using deep neural networks. Int J Comput Intell Syst. 2025;18(1). https://doi.org/10.1007/s44196-025-00783-x.
48. Alharbi F, Kashyap GS. Empowering network security through advanced analysis of malware samples: leveraging system metrics and network log Data for informed decision-making. Int J Netw Distrib Comput. 2024;12(2):250–64. https://doi.org/10.1007/s44227-024-00032-1.
49. Zhao Y, Sun S, Huang X, Zhang J. An android malware detection method using frequent graph convolutional neural networks. Electronics. 2025;14(6):1151. https://doi.org/10.3390/electronics14061151.
50. Vasilellis E, Anagnostopoulou A, Adamos K. Hidden realms: exploring steganography methods in games for covert malware delivery. In: Gritzalis D, Choo KKR, Patsakis C, editors. Malware. Advances in information security. Vol. 91. Cham: Springer; 2025. https://doi.org/10.1007/978-3-031-66245-4_15.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.